Remit-Scout Engineering (RSE)

Owner	@Omar Ghabayen
Status	Active
Last Reviewed	Dec 22, 2025
Environment	Production / Staging
Repositories	rilical/Remit-Scout-V2-Frontend

Description

Remit-Scout Engineering (RSE) is the authoritative knowledge base for the design, operation, and governance of Remit-Scout's data, backend, and infrastructure systems.

It documents architectural decisions, data-collection constraints, derived-data rules, APIs, and operational runbooks to ensure correctness, auditability, and long-term defensibility.

Quick Links

Repositories

- API Service: rilical/Remit-Scout-V2-Frontend
- Collectors: TBD
- Infra / IaC: TBD

Environments

- Production: TBD
- Staging: TBD

Dashboards

- API latency & errors: TBD
- Pipeline freshness: TBD
- ClickHouse health: TBD
- Aurora health: TBD
- WAF blocks / rate limits: TBD

Accounts & Consoles

- AWS Console: https://console.aws.amazon.com/
- Supabase: Supabase
- Stripe: https://dashboard.stripe.com/
- SES: https://console.aws.amazon.com/ses/

Documentation Index

Page	What it covers
ENG Home	Dashboards/repos/env links
Architecture Overview	System diagram, data boundary, service map, invariants
API Contracts	B2C/Plus/Telemetry/Marketing/Stripe schemas
Data Model	Aurora schema, QuoteRecord, Gold tables, FX feed

Pipelines	Scheduler/budgets, collectors, normalizer, gold agg, alerts eval
Operations	SLOs, runbooks, backup/restore, release checklist
Security & Compliance	safe collection protocol, rights matrix, threat model, WAF/rate limiting
Admin Console Spec	stoplist/budgets/circuit breakers/bans/re-aggregate/export retries
Decision Log (ADRs)	chronological decisions
Changelog	what changed / when / why

Product Backbone Modules (what the frontend depends on)

Module	Owns	Key APIs	Key tables
Identity + Billing + Entitlements	Supabase JWT + Stripe	/api/me, /api/billing/*	user_plan, plan_usage_counters
Watchlists + Alerts + Notifications	Plus features	/api/watchlist, /api/alerts	watchlist_item, alert_rule, alert_state, alert_event, notification_pref
History + Exports	Dashboard + B2B packs	/api/history/, */api/exports/*	export_job + ClickHouse gold_export.*
FX Mid-Market Rate Service	RCI correctness	internal FX	FX tables (Aurora + ClickHouse)

Telemetry Ingestion	Intent indices + UX	/api/telemetry/*	telemetry_search_event, telemetry_outbound_click
Contact + Newsletter	Growth ops	/api/contact, /api/newsletter/*	contact_submission, newsletter_subscriber

🏠 RSE Home

Executive Summary

Remit-Scout is a hybrid data platform with two distinct missions:

- **B2C Watchdog:** Provide fast, stable, consumer-facing comparisons (and "Pulse") via public APIs, without exposing internal raw artifacts or unsafe bulk outputs.
- **B2B Intelligence:** Produce institutional-grade **Derived Data** (indices, volatility signals, event markers) with provenance and auditability, while remaining compliant with third-party data rights.

This space documents **what we built**, **why it's built this way**, and **how to extend it without breaking its guarantees**.

Quick Navigation

- 🏛️ **Compliance & Governance (The Legal Firewall)** — policy, rights, publishing gate
- 🏗️ **System Architecture (The Machine)** — planes, services, dataflow, boundaries
- 💾 **Data Models & Schemas (The Map)** — Silver schema, Gold PIT model, FX model
- 📊 **Methodologies (The Alpha)** — how we compute indices & signals
- 🛠️ **Operations & Runbooks** — deploy, monitor, backfill, incident response
- 🧭 **Architecture Overview** — diagrams & infrastructure topology (AWS map, networks, IAM boundaries)

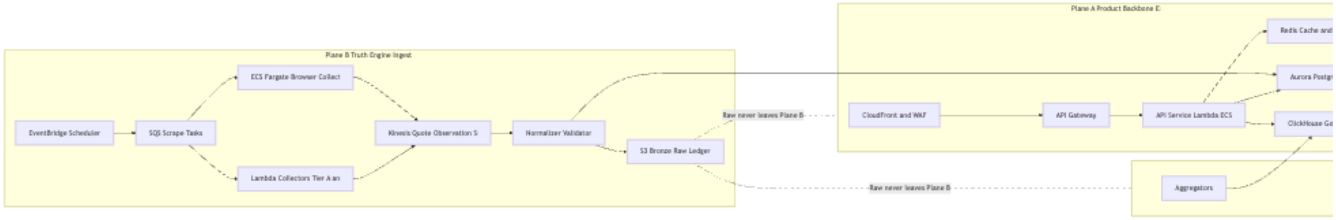
The "Golden Rule" of Data Flow (Pinned)

🚨 **Golden Rule: Raw data never leaves Plane B. Only Derived Data (Gold) leaves via Plane C.**

- "Raw" includes any provider-identifiable artifacts (HTML snapshots, raw responses, full quote streams) unless explicitly licensed (Tier A).
- Any B2B deliverable must be **Gold-only** and must pass the **Gold Publisher** gate (N≥3, dominance checks, rounding, provenance).

If you remember nothing else, remember this.

The 3-Plane Architecture (At a Glance)



Plane A — Product Backbone (External Surface Area)

Purpose: Serve users safely and reliably.
Typical components: CloudFront/WAF, API Gateway, Lambda APIs, Supabase Auth, Stripe billing webhooks, SES notifications, Redis cache.

Plane B — The Truth Engine (Ingestion + Normalization + Operational Truth)

Purpose: Collect and normalize quotes under strict compliance constraints.
Typical components: Collectors (Tier A API / Tier B headless), scheduler, stoplists & budgets, Bronze (S3), Silver (Aurora Postgres).

Plane C — Data Products (Aggregation + Publishing + Exports)

Purpose: Turn operational truth into **irreversible derived datasets** and deliver them.
Typical components: Aggregators, Gold (ClickHouse), export workers, sharing/sync mechanisms.

Storage Layers (Bronze / Silver / Gold)

- **Bronze (S3) = Immutable Raw Ledger**
  - Write-once, append-only.
  - Stores raw artifacts and raw quote payloads + provenance.
  - Used for replay, audit, and backfill (never for external delivery).
- **Silver (Aurora Postgres) = Operational Truth**
  - Normalized quote records + product state (users, plans, alerts, exports, telemetry).
  - Optimized for low-latency reads/writes and correctness.
  - Retention is operationally bounded; Bronze remains the long-term record.
- **Gold (ClickHouse) = Derived Data (Commercial Layer)**
  - Aggregates, indices, signals, events.
  - Designed for time-series analytics and B2B deliverables.
  - **Must** maintain PIT semantics and restatement rules.

Why It Was Built This Way

1) Compliance by Construction

We separate ingestion (Plane B) from delivery/publishing (Planes A/C) so that **even a bug** in an API cannot leak raw data. Boundaries are enforced by:

- Credential scoping (IAM roles, DB users)
- Network segmentation (private subnets/security groups)
- Code paths (no endpoints that can return raw artifacts)
- Publishing gates (Gold Publisher protocol)

2) Auditability & Provenance

We keep an immutable Bronze record so we can always answer:

- “What did we observe?”
- “When did we observe it?”
- “How was this derived?”
  This is essential for investor diligence, compliance defense, and debugging.

3) Scalability + Reliability

We use serverless/stateless delivery (Plane A) and isolate heavy collection (Plane B) and analytics (Plane C), so spikes in traffic or a broken collector do not take down the product.

4) Future-proofing: PIT + Restatements

Gold is designed to support “as-of” queries and restatements without rewriting history—critical for institutional workflows.

Core Guarantees (Non-Negotiables)

Compliance & Rights

- No login scraping. No CAPTCHA solving farms. No bypassing clickwrap/ToS prompts.
- **Stop on block**: if blocked, we back off and stop attempts (no evasion loops).
- B2B outputs are derived + irreversible (N≥3 minimum + dominance checks + rounding).

Security

- Least privilege everywhere; services can only access what they must.
- Raw Bronze is never readable from user-facing services.

Data Quality

- All published numbers have provenance and algorithm versioning.

- “No data” is preferable to fabricated data.
- Anomaly detection prevents “fat finger” outputs from being published.

PIT Correctness

- Gold facts are bitemporal (reference_time vs knowledge_time).
- Restatements are **append-only** (insert new version; do not overwrite history).

How To Modify or Extend Safely (Engineer Checklist)

Before you ship changes, answer these:

1) Which plane am I changing?

- **Plane A:** APIs/auth/product features → verify you cannot reach Bronze; verify response schema contains only allowed fields.
- **Plane B:** collectors/normalization → verify stop-on-block, budgets, rights matrix alignment; ensure provenance links are written.
- **Plane C:** aggregators/publishing/exports → verify Gold Publisher gate, PIT semantics, restatement policy, and external delivery rules.

2) Did I preserve the Golden Rule?

- No new code path that could export raw artifacts.
- No B2B dataset generated from anything except **Gold**.

3) Did I update governance artifacts?

- Data Rights Matrix updated (if providers/permissions changed).
- ADR written (if methodology/schema/publishing rules changed).
- Runbooks/alerts updated (if operational behavior changed).

4) Did I add or adjust monitoring?

- Freshness: “Did corridor/provider update today?”
- Anomalies: “Did a provider move 90% in 1 hour?”
- Pipeline health: collector failures, block detection, aggregator lag.

5) Did I test the failure modes?

- Provider blocked → collector stops + stoplist/circuit breaker engages.
- Partial data → published output withheld or marked “insufficient contributors.”
- Backfill/replay works from Bronze for the modified schema/method.

Glossary (Working Terms)

- **Plane A:** Product backbone and APIs (external surface area)
- **Plane B:** Truth Engine / ingestion and operational truth
- **Plane C:** Data Products / publishing and exports
- **Bronze:** Immutable raw ledger (S3)
- **Silver:** Normalized operational database (Aurora Postgres)
- **Gold:** Derived analytical data (ClickHouse)
- **PIT:** Point-in-time; bitemporal modeling (reference_time vs knowledge_time)

Closing Note

This documentation exists so future engineers (and AI agents) can understand not just *what* the system does, but **why the separations exist** and **what must never be broken**. With these foundations, the platform is well-equipped to adapt and grow while maintaining the integrity, security, and reliability that are paramount to its mission.

🏗️ System Architecture

This page is the *technical blueprint* for how Remit-Scout is built and how the planes interact. It intentionally keeps **Ingestion** separate from **Delivery** to reinforce the mental model of the firewall.

Architecture Index & Execution Docs

This page is the mental model + boundary contract (Planes A/B/C + guarantees).
For planning, implementation, and LLM-assisted task generation, use the catalogs and service pages linked below.

Catalogs (required for planning + tech debt control)
Create these as child pages under 🏗️ System Architecture and treat them as the "index of record":

- ***Service Catalog (what exists, who owns it, how it runs)***
- ***API Catalog (endpoints, auth, entitlements, source-of-truth OpenAPI link)***
- ***Event & Queue Catalog (SQS/Kinesis/EventBridge, schemas, DLQs)***
- ***Data Stores & Retention Catalog (Bronze/Silver/Gold + retention + access roles)***
- ***SLOs & Monitoring Catalog (SLIs, alerts, dashboards, runbooks)***
- ***ADR Index (architecture decisions with dates + rationale)***
- ***LLM Planning Playbook (how to generate epics/stories/tasks consistently)***

Documentation Definition of Done (DoD)
A change is "documented" only when the matching catalog(s) are updated:

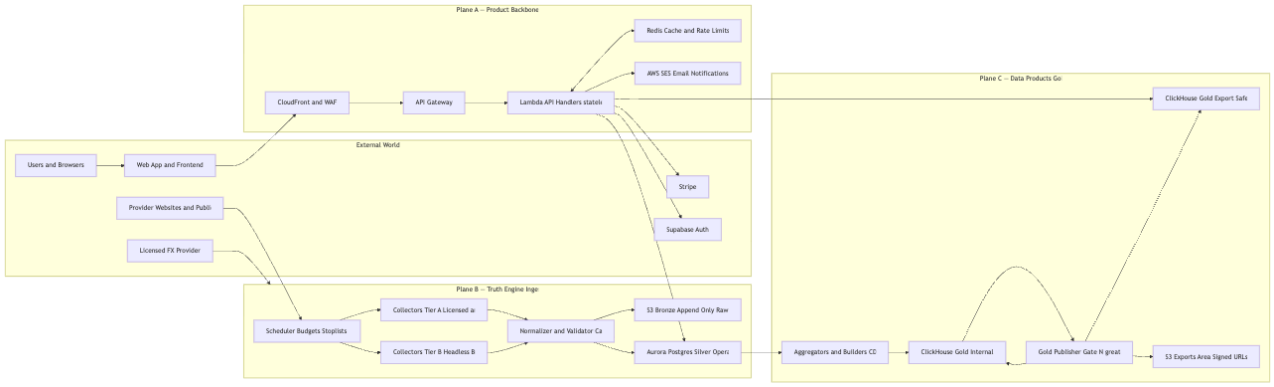
- ***New/changed service → Service page + Service Catalog entry + owner + runbook link***
- ***New/changed endpoint → OpenAPI updated + API Catalog row + auth/entitlement classification***
- ***New/changed queue/event/job → Event Catalog row + schema version + DLQ policy***
- ***New/changed table → migration + Data Models entry + retention + access role notes***
- ***Any change affecting data boundary / publishing rules → ADR + Publisher gate tests updated***

Architecture Principles

- **Separation of concerns**
  - Plane A serves product experiences
  - Plane B observes and normalizes “truth”
  - Plane C derives + publishes commercial datasets
- **Least privilege**
  - Each component can only touch the data it must
- **Auditability**
  - Every quote can be traced to a Bronze record + ingestion run
- **Fail-safe defaults**
  - *Withhold over publish*; *stop over evade*
- **Non-negotiables**
  - No login scraping / account creation / clickwrap acceptance
  - No CAPTCHA solving farms / bypass tactics / “undetected” evasion
  - If blocked or revoked: **stop collection** + escalate (partnership / review)
  - B2B outputs: **derived only** with irreversibility gates (N≥3 + suppression + rounding + bucketing)

System Overview — The 3-Plane Model

> **Golden Rule (pinned concept):** Raw artifacts never leave Plane B. Only **Derived Data (Gold Export)** leaves via Plane C.



Interpretation

- Plane B captures truth and writes it to **Bronze + Silver**.
- Plane C reads **Silver**, derives analytics, and publishes only to **gold_export** after gates.

- Plane A serves the app by reading **Silver "current"** and **Gold "derived"**—and is structurally unable to access raw Bronze.

2.1 Plane A — Product Backbone (Delivery Surface)

Responsibilities

- Provide stable APIs for consumer experiences (compare pages, Pulse)
- Manage identity, billing, and entitlements (Supabase + Stripe)
- Deliver user features (watchlists, alerts, exports requests)
- Enforce rate limiting + WAF protections + abuse controls

Typical Components

- **Edge**: CloudFront + WAF
- **Routing**: API Gateway
- **Compute**: Lambda (stateless)
- **Cache / RL**: Redis (short TTL, non-source-of-truth)
- **Auth**: Supabase (JWT verification)
- **Billing**: Stripe (webhooks)
- **Notifications**: SES

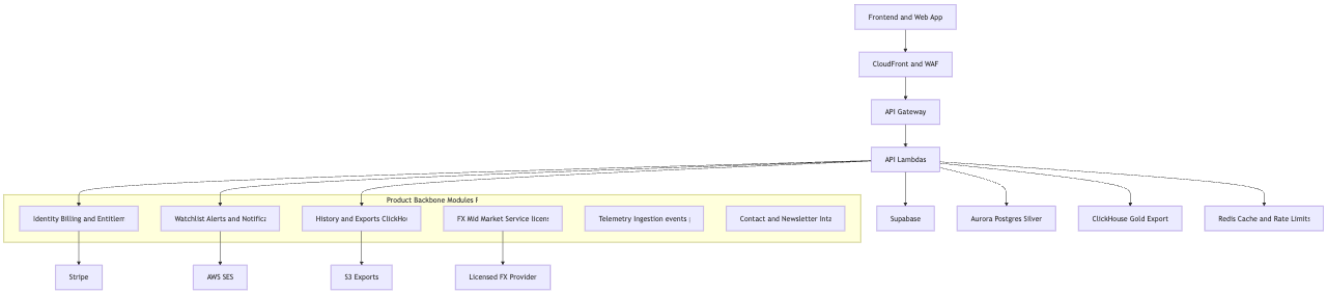
The missing layer — Product Backbone Services (first-class modules)

These are *not optional*. They are the modules the frontend expects and the "app" needs beyond ingest + lakehouse.

- **Identity + Billing + Entitlements**
  - Supabase JWT verification
  - Stripe checkout + webhooks
  - Plan enforcement + quotas (Free/Plus/B2B)
  - `/api/me` as the single source of truth for plan + entitlements
- **Watchlist + Alerts + Notifications**
  - Persist watchlists & alert rules
  - Alert evaluation workers (daily/hourly/realtime as allowed)
  - Alert history, notification prefs, unsubscribe handling
  - Guest alerts (double opt-in)
- **History + Exports**

- Time-series endpoints backed by Gold (ClickHouse)
- Export job queue → file generation (CSV/PDF) → S3 signed downloads

- **FX Mid-Market Rate Service**
  - Ingest licensed mid-market feed
  - Store aligned rates for QuoteRecord computation + charting
- **Telemetry Ingestion**
  - Persist searches/clicks (privacy-safe)
  - Feed “Intent” indices later (with privacy gating)
- **Contact + Newsletter Intake**
  - Store submissions
  - Double opt-in for marketing/newsletter lists



Guarantee (Plane A)

Plane A services must not have credentials or network paths to Bronze (raw artifacts).

Plane A can:

- read **Silver** for “current operational truth”
- read **gold_export** for “safe derived time series”
- never read **Bronze** and never bypass Publisher gates

2.2 Plane B — The Truth Engine (Ingestion)

Responsibilities

- Collect public quotes safely and ethically
- Persist immutable raw ledger (Bronze)
- Normalize into operational truth (Silver)
- Enforce stoplists, budgets, and circuit breakers

Collector Specs

- **Tier A (API / Licensed)**

- Partnered / explicitly permitted sources
- Rights recorded in the Rights Matrix

- **Tier B (Headless Browser / Public UX)**
  - Emulates a normal user flow
  - Never logs in, never accepts clickwrap prompts, never solves CAPTCHAs
  - Stops immediately on block signals

Scheduling & Budgets

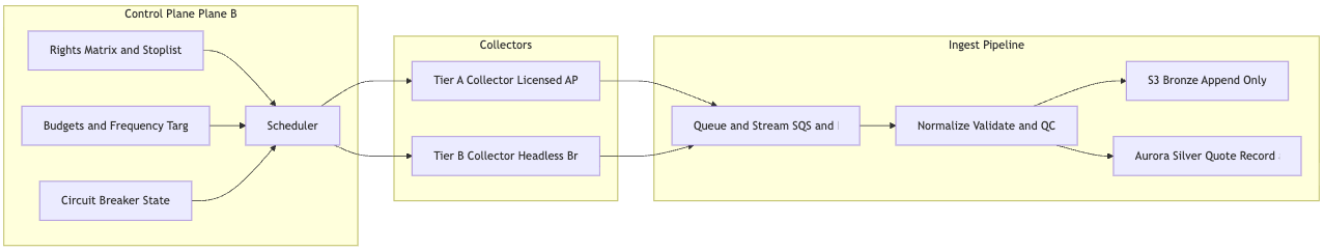
- Collection frequency is budgeted per provider and corridor
- Any deviation requires explicit approval (cost + compliance impact)
- Circuit breakers prevent “thrash” and enforce **stop over evade**

Proxy Strategy (Localization-only)

- Country targeting is allowed to observe local variants of public pricing
- Not allowed for evasion: if blocked, stop and escalate

Normalization Rules

- Provider labels → canonical IDs (e.g., “Western Union” → `provider_id=wu`)
- Currency codes normalized (ISO-4217)
- Amounts/rates stored with precise numeric types + explicit scale
- Every Silver row traces to Bronze provenance (ingestion_run / record key)



Outputs

- Bronze: append-only raw record + provenance
- Silver: normalized QuoteRecord + ingestion metadata + “latest” materialized tables

2.3 Plane C — Data Products (Aggregation + Publishing for B2B)

Responsibilities

- Generate and store derived datasets (Gold)
- Enforce publishing protocol (“Gold Publisher”)
- Produce exports and controlled sharing channels

Data Products (examples)

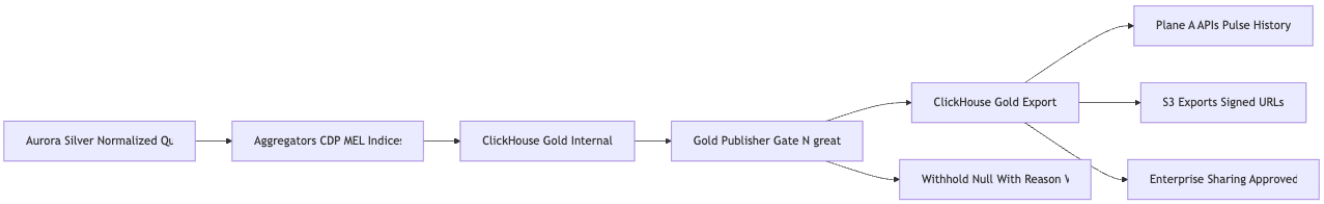
- Corridor indices (RCI, volatility signals, event markers)
- PIT-safe time series (bucketed)
- Client exports (CSV/Parquet/PDF) generated by export workers

Gold Publisher (mandatory gate)

Publisher is the *only* way data becomes externally consumable.

Rules (high level)

- **N≥3** independent providers per published aggregate
- **Dominance checks** (withhold if one provider dominates and could be inferred)
- **Timestamp bucketing** (4h / daily for exportable datasets)
- **Rounding / suppression** (reduce reversibility; withhold on small-N)
- **Audit metadata** (contributor_count bucket, freshness stats, methodology version)



Guarantee (Plane C)

- Anything leaving the platform (exports, B2B delivery, externally queryable datasets) must originate from **gold_export** and must pass Publisher gates.

Cross-Plane Boundaries (Enforcement)

Technical Controls

- IAM roles scoped per plane/service

Dynamic Sentinel Protocol (Per-Corridor)

Goal

Detect meaningful market changes quickly without relying on a single global provider, and remain compliant with stop-on-block.

Sentinel election

- For each corridor, the system elects a primary sentinel provider based on recent reliability and responsiveness (success rate, latency, stability).
- A secondary sentinel provider is also stored as backup.

- Election runs on a scheduled cadence (e.g., weekly) and produces a corridor → (primary, backup) mapping.

Failover

- If a scheduled fetch to the primary sentinel fails, immediately retry using the backup sentinel.
- Failover events are logged and surfaced to monitoring.
- If the failure is a block signal (403, CAPTCHA, explicit anti-bot), enforce stop-on-block: mark provider unhealthy for that corridor, place on cooldown/stoplist, do not retry aggressively.

FX-triggered scrapes

- When a corridor's mid-market reference rate moves beyond a configured threshold, trigger immediate fetches for that corridor using the corridor's elected sentinel (not a global sentinel).
- This improves responsiveness while distributing load and avoiding a single point of failure.

Low-coverage corridors

- If no provider meets the reliability bar (or provider count is too low), the system may: rely primarily on FX-triggered events for that corridor, and/or use a documented proxy indicator corridor for volatility detection (last resort).
- Network segmentation (private subnets; security groups)
- Separate DB users/permissions by workload
- S3 bucket policies: **no Plane A access to Bronze**

Code Controls

- No endpoints returning raw artifacts
- Publisher gate is a mandatory step for any dataset classified as "external"
- Fail-safe behavior: *withhold* over publish; *stop* over evade

What belongs in "Architecture Overview"

This page stays at the **mental-model + service boundary** level. Detailed infrastructure diagrams (AWS resource names, VPC CIDRs, IAM policy snapshots, Terraform/CDK module references) live in **Architecture**

📚 Service Catalog

Purpose: Lists all major services/components, including which plane they belong to, what they do, who owns them, and how they run. This helps identify impact of changes by showing **what exists, who owns it, and how it runs** in one place.

Service Name	Plane	Description	Owner	Runtime / Notes
Identity & Billing Service	A (Product)	Manages user identity/auth (Supabase), subscription entitlements, and billing (Stripe). Ensures `/api/me` returns source of truth for user plans. Enforces plan quotas at runtime.	Product Eng.	Serverless (AWS Lambda + API Gateway). Uses Supabase (Postgres) for auth and Stripe webhooks.
Watchlist & Alerts Service	A (Product)	Handles user watchlists and price alerts. Persists watchlist items and alert rules; runs alert	Product Eng. (Plus)	Serverless functions (scheduled invocations for alert checks). Uses Aurora (Silver DB) for state +

		evaluation jobs (daily/hourly or real-time triggers) to notify users. Includes email notifications via SES with unsubscribe + guest opt-in flows.		SES for email.
History & Exports Service	A (Product)	Provides time-series data and export functionality. Serves historical rate data backed by Gold-derived datasets. Handles export job requests (CSV/PDF), queues jobs, produces files on S3.	Product Eng. / B2B	Serverless (on-demand). Uses ClickHouse (Gold) for time-series + Export Job Lambda (triggered via SQS).
FX Mid-Market Rate Service	A (Product)	Ingests a licensed FX reference feed to provide a	Data Eng. (Platform)	Scheduled ingestion (cron). Writes to Silver/Gold

		baseline mid-market rate. Aligns/stores rates for quote calcs and charts (common baseline for measuring fees).		(Aurora & ClickHouse).
Telemetry Collection Service	A (Product)	Collects telemetry (search queries, outbound clicks) in a privacy-safe way. Stores only non-PII identifiers and basic event info under strict privacy gating.	Growth / Eng.	Public API POST endpoints. Writes to Silver (Aurora) telemetry tables. Enforces data minimization.
Contact & Newsletter Intake	A (Product)	Captures contact form submissions and newsletter sign-ups. Implements double opt-in for marketing subscriptions	Growth / Eng.	Public API endpoints writing to Silver tables. Confirmation emails via SES (or third-party).

		; stores submissions for follow-up.		
Ingestion Orchestrator	B (Ingestion)	“Truth Engine” scheduler orchestrating collection runs. Budgets/triggers tasks per provider/corridor. Honors stoplists + circuit breakers. Ensures safe/ethical collection per compliance rules.	Data Eng.	AWS EventBridge Scheduler (cron) or Step Functions. Enforces rate limits and checks Data Rights Matrix before each run.
Headless Collector Lambdas	B (Ingestion)	Collect public quotes. Tier A uses official partner APIs; Tier B simulates a user in headless browser (no login, no CAPTCHA	Data Eng.	AWS Lambda (per provider or shared). Uses Playwright/Puppeteer for Tier B. No stored credentials. Stops on any block signal. Outputs

		solving, no ToS bypass). Writes raw results to Bronze and normalized records to Silver.		include provenance (run IDs, timestamps).
Normalizer & Silver Writer	B (Ingestion)	Parses raw artifacts (Bronze) into standardized `QuoteRecord` entries in Silver (Aurora). Applies normalization rules and attaches provenance (link to Bronze). Ensures every Silver quote traces to Bronze.	Data Eng.	Lambda or Fargate, event-driven. Writes Aurora tables. Idempotent upsert for “latest_quote” views.
Gold Aggregator & Publisher	C (Data Products)	Aggregates Silver → Gold datasets (indices, volatility, etc.). Enforces	Data Eng. (Analytics)	Batch jobs (daily/intraday). Uses ClickHouse. Withholds outputs that fail gates. Produces

		Gold Publisher Protocol gates (N≥3 providers, dominance checks, time bucketing, rounding/suppression). Writes approved outputs to `gold_export` tables.		audit logs for published vs withheld points.
Export Worker	C (Data Products)	Generates export files (CSV/Parquet/PDF) from Gold on-demand. Ensures only Gold-approved data is exported (no raw/Silver).	Product Eng. / B2B	Lambda via queue. Writes files to S3; provides signed URLs. Post-generation checks (derived data only).

References / Invariants

- Plane A services **cannot access** Plane B raw storage (no network path/credentials to Bronze). See: System Architecture and Golden Rule: Raw Data Stays Internal.
-

Additional Features and Decisions

Pulse (Opportunity Ticker) Feature

- *Feature definition*: Pulse is a real-time opportunity ticker that highlights corridors where the current transfer rate is significantly better than its recent average.
- *Logic and query*: For example, we flag quotes where the current price is more than 5% cheaper than the 7-day average (SQL: `SELECT * FROM quotes WHERE current_price < 0.95 * average_price_last_7_days`).
- *User value*: This surfaces deals like “show me corridors where fees are 5% lower than usual right now” and ties the technical query back to a user benefit.

Build-vs-Buy Decisions

- *Auth*: We use **Supabase Auth** for sign-up, login, password reset and email verification. No custom auth system is built; developers should configure Supabase email templates and JWT handling rather than rolling our own.
- *Payments*: All payment flows, including subscriptions and upgrades, rely on **Stripe Checkout**. We do not implement custom credit-card forms or payment logic.
- *Transactional Email*: We rely on **AWS SES/Postmark** for sending verification emails, alerts and notifications. This ensures deliverability and proper unsubscribe handling.
- *Proxy/Scraping*: For IP rotation in provider scraping, we use commercial proxy services (e.g., Bright Data or Smartproxy) instead of building an in-house network.

These choices are documented in our Architectural Decision Records (ADR) to ensure new team members understand why we rely on proven third-party services and to avoid re-litigating these decisions.

- Plane B follows strict compliance (no evasion; stop-on-block). See: No Evasion / Stop-on-Block Policy.
- Plane C enforces publisher gates for any data leaving the platform. See: Gold Publisher Protocol.

📜 API Catalog

Purpose: Enumerates public/internal API endpoints including authentication, entitlement requirements, and source-of-truth schemas (OpenAPI). Ensures every API has a clear contract (path, method, auth, rate limits/plan levels).

API Endpoint	AuthN / Entitlement	Description	Notes (OpenAPI / Source)
GET `/api/quotes` (core compare)	None (Public) (IP rate-limited)	Retrieve current best quotes for a corridor (send X amount from A→B). Powers B2C comparisons while never exposing raw provider data (derived-only).	OpenAPI: public.yaml. Outputs derived "latest quote" views from Silver (no Bronze).
GET `/api/me`	User JWT (Auth) (All plans)	Returns authenticated user profile, plan tier, entitlements. Source of truth for client apps re: features.	OpenAPI: users.yaml. Reads from Silver user/plan usage tables.
POST `/api/watchlist` / GET `/api/watchlist`	User JWT (Plus required)	Create/list watchlist items. Stored in Silver and tied to user.	OpenAPI: product.yaml. Enforces Plus entitlement.

POST `/api/alerts` / GET `/api/alerts`	User JWT (Plus required)	Create/list alert rules (e.g., “notify me if USD→EUR < X”). Backend jobs evaluate triggers; endpoint manages config.	OpenAPI: product.yaml or alerts.yaml. Endpoint manages config in Silver tables.
GET `/api/history/{corridor}`	User JWT (Plus or B2B)	Historical time-series for corridor (daily indices/rates). Backed by Gold export data in ClickHouse (derived aggregates).	OpenAPI: history.yaml. Returns from `gold_export` tables (publisher-safe).
POST `/api/exports`	User JWT (Plus/B2B)	Request export file (CSV/PDF). Enqueues export job; returns job ID or download link when ready.	OpenAPI: exports.yaml. Produces message to `ExportJobQueue` for Export Worker.
POST `/api/telemetry/search` / POST `/api/telemetry/click`	None (Public) (anon ID)	Captures frontend telemetry events (searches, outbound clicks). Payload sanitized (no PII); stored for analytics.	OpenAPI: public.yaml. Writes to Silver telemetry tables. Strict size/rate limits.
POST `/api/conta`	None (Public)	Contact/support form submission.	OpenAPI: public.yaml.

`ct`		Stores inquiry; triggers confirmation email.	Writes to Silver `contact_submission`. SES confirmation email.
POST `/api/newsletter/subscribe`	None (Public)	Newsletter signup + double opt-in flow (user must confirm via email link).	OpenAPI: public.yaml. Writes to Silver `newsletter_subscriber`. Confirmation via SES.

References / Invariants

- Consumer-facing endpoints **never leak raw provider artifacts**; public data is operational (Silver) or derived (Gold export). See: Golden Rule: Raw Data Stays Internal and Gold Publisher Protocol.
- Entitlements are enforced server-side (Free vs Plus vs B2B). See: Entitlements & Plans Spec.
- OpenAPI specs are source of truth; update them whenever endpoints change. See: API Documentation Policy.

Frontend-Observed Endpoints (Audit)

Pulse Analytics

	Method		Path		Auth		Entitlement		Description		Notes	
GET	/api/pulse/overview	User JWT	Plus/Enterprise	pulse overview	OpenAPI: pulse.yaml
GET	/api/pulse/chart/{id}	User JWT	Plus/Enterprise	chart data by ID	OpenAPI: pulse.yaml
GET	/api/pulse/events	User JWT	Plus/Enterprise	events feed (optional)	OpenAPI: pulse.yaml
GET	/api/pulse/providers/benchmarking	User JWT	Plus/Enterprise	provider benchmarking	OpenAPI: pulse.yaml
GET	/api/pulse/method-coverage	User JWT	Plus/Enterprise	method coverage	OpenAPI: pulse.yaml

Core Compare & Public Discovery

	Method		Path		Auth		Entitlement		Description		Notes	
GET	/api/quotes/current	None	Public	core comparison (best quotes)	OpenAPI: public.yaml
GET	/api/providers	None	Public	alias of core compare	use /api/quotes/current
GET	/api/popular-corridors	None	Public	popular corridors	OpenAPI: public.yaml
GET	/api/recent-searches	User JWT	Free & Plus	recent user searches	OpenAPI: product.yaml
POST	/api/recent-searches	User JWT	Free & Plus	record search	OpenAPI: product.yaml
GET	/api/bank-vs-specialist	None	Public	bank vs specialist summary	OpenAPI: public.yaml
GET	/api/geo	None	Public	geolocation data	OpenAPI: public.yaml

Billing

	Method		Path		Auth		Entitlement		Description		Notes	
POST	/api/billing/checkout-session	User JWT	Plus upgrade	create Stripe checkout session	OpenAPI: billing.yaml
POST	/api/billing/webhook	None	N/A	Stripe webhook handler	OpenAPI: billing.yaml
GET	/api/billing/portal	User JWT	Plus	Stripe customer portal link	OpenAPI: billing.yaml

Telemetry & Contact

	Method		Path		Auth		Entitlement		Description		Notes	
POST	/api/telemetry/search	None	Public	search telemetry	OpenAPI: public.yaml
POST	/api/telemetry/click	None	Public	click telemetry	OpenAPI: public.yaml
POST	/api/contact	None	Public	contact form submit	OpenAPI: public.yaml
POST	/api/newsletter/subscribe	None	Public	newsletter subscription	OpenAPI: public.yaml

Pulse Analytics

Overview & Purpose

Pulse is a premium analytics feature that surfaces opportunities and trends in remittance corridors. It answers questions like “Is now a good time to send money on USD→MXN?” by comparing current costs to historical averages and highlighting notable patterns. Pulse contains charts and widgets showing cost trends, volatility, provider performance and coverage.

API Endpoints

The frontend calls a set of endpoints, which require a logged-in user with the Plus (or enterprise) plan:

- `GET /api/pulse/overview` – returns top-level summary of current opportunities and high-level stats.
- `GET /api/pulse/chart/{id}` – returns dataset for a specific Pulse chart or widget.
- `GET /api/pulse/events` – optional feed of notable market events.
- `GET /api/pulse/providers/benchmarking` – comparative statistics on providers for a corridor.
- `GET /api/pulse/method-coverage` – comparison of payment method coverage and costs.

Each endpoint corresponds to an OpenAPI spec (see pulse.yaml) and is subject to rate limiting and caching.

Chart & Metric Definitions

Pulse includes many visualizations, each defined by a dataset and metrics from our Gold warehouse:

- **All-In Cost Trend** – time series of the cheapest all-in cost (RCI) on a corridor.
- **Cost Volatility** – measure of day-to-day variability in exchange rates or RCI.
- **Leader Change Frequency** – how often the cheapest provider changes over time.

- **Provider Availability & Benchmarking** – share of time each provider is active and how often it is cheapest or fastest.
- **Coverage Summary** – breakdown of pay-in/pay-out method availability across providers.
- **Smart Send Savings** – potential savings between the cheapest and most expensive providers.

Additional widgets may include arbitrage indicators or historical exchange vs paid rates.

Opportunity Definition

Pulse flags a corridor as an **opportunity** when the current best all-in cost is significantly lower than its recent average. For example, if the current RCI is at least 5% lower than the 7-day average, the corridor is listed in the opportunity overview. This threshold is tunable.

Entitlement & Access

Pulse data is available only to authenticated users on Plus or enterprise plans. Free-tier users cannot call the Pulse endpoints. The server validates plan entitlements on each request.

Data Sources & Compliance

- Pulse metrics are derived from aggregated data stored in our Gold warehouse.
- All outputs adhere to the Gold Publisher Protocol: at least 3 providers contribute to any statistic, dominance checks are enforced, and values are rounded to avoid revealing raw quotes.
- Data refreshes periodically (e.g., hourly) rather than in real time to reduce system load and ensure compliance.
- No individual provider's live quotes or sensitive data are exposed.

Entitlements & Plans

Plan Tiers & Feature Access

- **History Range:** Free – 30 days; Plus – 365 days; Enterprise – Extended history
- **Watchlist Items:** Free – up to 3; Plus – up to 50; Enterprise – unlimited
- **Price Alerts:** Free – none; Plus – up to 5 active; Enterprise – higher/unlimited
- **Data Exports:** Free – not available; Plus – CSV/PDF export; Enterprise – enabled
- **Pulse Analytics:** Free – not available; Plus – full access; Enterprise – full access
- **API Rate Limits:** Free – standard; Plus – higher; Enterprise – highest

Note: The exact quotas may change. Check the UI for current values.

/api/me Entitlement Response

The `/api/me` endpoint returns a JSON object with the user's profile, plan, and entitlements. Important entitlement fields include:

- **historyMaxDays** – maximum days of history the user can query
- **alertsMax** – maximum active price alerts
- **watchlistMax** – maximum watchlist items
- **exportsEnabled** – whether data exports are allowed
- **pulseAccess** – whether Pulse analytics are accessible

The server uses these values to enforce plan gating on each request.

Plan Enforcement & Billing Integration

- Authentication and plan metadata are managed via Supabase Auth and Stripe.
- To upgrade to Plus, the client calls `POST /api/billing/checkout-session` to create a Stripe Checkout session; Stripe webhooks update subscription status.
- When a subscription lapses or is canceled, the user's `planStatus` becomes inactive and premium entitlements are revoked.
- The frontend should call `/api/me` on load and after any billing events to refresh entitlements.

🔔 Event & Queue Catalog

Purpose: Documents async events, queues, and streams across planes—message schemas, producers/consumers, DLQs, retry policies.

Event/Queue Name	Type	Purpose & Schema	Producer	Consumer	DLQ / Notes
CollectionTaskQueue	SQS Queue	Dispatches quote-collection jobs (provider + corridor + amount tier). Fields: `provider_id`, `corridor_id`, run context (`schedule_id`, etc.). Decouples scheduling from execution.	Ingestion Orchestrator (Plane B)	Headless Collector Lambda	Yes: `CollectionTaskQueue-DLQ`. Monitor DLQ spikes (widespread collection failures).
ExportJobQueue	SQS Queue	Export generation jobs.	`/api/export`	Export Worker (Plane C)	Yes: `ExportJobQue`

		Fields: `export_job_id` + parameters (user, dataset, time range).	`s` (Plane A)		`ue-DLQ`. Runbook for stuck exports.
FX Rate Trigger & Decoupled Scraping The collection pipeline supports an on-demand trigger based on foreign-exchange movements. An external mid-market FX feed is monitored; if a currency moves by more than 0.5% within 10 minutes,	EventBridge Schedule	Triggers daily Gold aggregation pipeline (nightly cron). Payload: date/window to aggregate.	EventBridge Rule	Gold Aggregator job (Plane C)	No DLQ (EventBridge/Lambda retries). Monitor lag/staleness alarms.

the system immediately enqueues scrape tasks for all Tier-1 corridors involving that currency. These tasks are dispatched via the **CollectionTaskQueue** (an SQS queue), ensuring that scrapes are always decoupled from user API calls. A fleet of headless collector Lambdas consumes the queue asynchronously, fetches provider						

quotes using proxy rotation, and writes results to the database. This architecture prevents blocking user requests and allows horizontal scaling. Use DLQ metrics to monitor for bursts of failed tasks (e.g., provider blocks), and leverage retry/backoff policies accordingly.DailyGoldAggregation					
Telemetry Stream (Planned)	Kinesis Stream	High-volume telemetry for near	Telemetry Service (Plane A)	Streaming consumer (Plane C)	No DLQ (Kinesis retention). Monitor

		real-time analytics/ ML. Payload: anonymized event JSON.			consumer lag (iterator age).
AlertTriggerEvent	EventBridge Schedule	Triggers periodic alert evaluation runs (e.g., every 5 min / hourly / daily). Payload: timestamp or none.	EventBridge schedules	Alerts Engine	No DLQ. Monitor stale alert metrics + failures in logs.

Notes

- Default strategy: decouple via queues, always define schema + retry/DLQ behavior. See: Async Messaging Standards.
- All SQS queues should have DLQs + alarms. See: Queue Monitoring Standard.

🗄️ Data Stores & Retention Catalog

Purpose: Documents all data stores, retention policies, and access controls. Emphasizes Bronze → Silver → Gold lifecycle plus supporting stores.

Data Store (Layer)	Technology	Purpose / Content	Retention Policy	Access Control & Roles
Bronze (Raw Ledger)	AWS S3	Immutable storage of raw ingestion artifacts/payloads + metadata. Used for audit, replay, backfills. Never served to users.	Indefinite (append-only). Purge only via explicit legal/compliance process.	Write: Plane B Collectors. Read: Plane B internal only. Plane A has **no access** (IAM + bucket policy).
Silver (Operational DB)	Amazon Aurora Postgres	Normalized operational DB: processed quotes + all user-facing state (users, watchlists, alerts, telemetry, etc.). Transactional.	Bounded operational window (e.g., keep last 6–12 months). Older history consolidated into Gold; Bronze preserves full audit history.	Write: Plane B (normalized quotes) + Plane A (product state). Read: Plane A (current latest) + Plane C (aggregation). Fine-grained roles.

Gold (Analytical DB)	ClickHouse	Derived warehouse for aggregates/time-series with PIT semantics + versioning. Split: `gold_internal` vs `gold_export` (externally safe).	Indefinite (append-only). Restate via version entries rather than delete.	Write: Plane C jobs only. Read: Plane A reads `gold_export` only; Plane C can read internal + export. External access always from `gold_export`.
Supabase Auth DB (Users)	Supabase Postgres	Auth store (separate from Aurora). Credentials + minimal PII for auth. JWT verification source.	Indefinite until user deletion request. Backups per Supabase defaults.	Plane A only (Identity service). Not accessible to Plane B/C.
Redis Cache (Ephemeral)	AWS ElastiCache Redis	Cache/rate-limiter/session/transient data. Not source of truth.	TTL-based seconds→minutes. Evicted on TTL/memory pressure.	Plane A services only. No persistent business data.

Invariants

- Every Silver record must trace to a Bronze artifact (provenance).
- No API should ever require Bronze access. See: Golden Rule: Raw Data Stays Internal.

📈 SLOs & Monitoring Catalog

Purpose: Defines key SLOs and how they're measured + monitored (SLIs, alerts, dashboards, runbooks). Covers both product reliability and pipeline freshness.

Service / Metric SLI	SLO Target	Monitoring & Alerts	Dashboard / Runbook
B2C API Availability (Public APIs)	≥ 99.9% uptime (monthly)	Synthetic checks on core endpoints + alarms on 5XX spikes/downtime > 1 min. Page on-call for SEV0 outage.	API Latency & Error Dashboard. Runbook: RBK-SEC-001: API outage or abuse.
Data Freshness – Top Corridors	p95 "last update" ≤ 15 min (Tier-1); Cold corridors ≤ 4 hours	Metric: age of latest Silver quote for Tier-1 corridors. Alarm if >5 min for 5% of checks or any corridor >10 min.	Pipeline Freshness Dashboard. Runbook: RBK-PIPE-001: Freshness breach.
Gold Aggregation Lag (Publishing)	Intraday lag ≤ 15 min; Daily ready by 02:00 UTC	Alarm if intraday job >15 min late or daily job >2 hrs late; alert on Gold Publisher job failures.	Gold Build Status Dashboard. Runbook: RBK-PUBLISH-001: Publisher gate failures.
Quote Collection Success Rate (Tier-1 providers)	≥ 98% success (per 1h window)	Ratio of successful runs vs errors/blocks. Alert on drops. Immediate alert	Ingestion Ops Dashboard. Runbook: RBK-COLLECT-001:

		on block signals (CAPTCHA/403 patterns).	Provider blocked.
User Alert Delivery (Notifications)	≥ 99% delivered within SLA window (excluding unsubscribes)	SES metrics for bounces/failures; alert on delivery drop/delay spikes.	Alerts/Email Dashboard. Runbooks: RBK-EMAIL-001: SES spike and RBK-ALERTS-001: Alert delivery issues.
Database Health – Aurora	≤ 70% CPU; minimal replica lag	Performance Insights + CloudWatch: CPU, connections, replica lag, failover events.	DB Health Dashboard. Runbook: RBK-DB-001: Aurora issues.
Data Quality – Anomaly DetectionAdditional SLOs **Search API Latency**: P95 latency under 200 ms for search API responses. Monitor p95 latency via API logs and metrics; alert when latency exceeds 200 ms for more than 5% of	0 critical anomalies in Gold outputs	Automated checks on new Gold points vs historical ranges. Anomalies trigger review; publishing gates likely withhold.	Anomaly Monitor Dashboard. Included in RBK-PUBLISH-001.

requests in a 5-minute window. Dashboard: API Latency & Error Dashboard. Runbook: RBK-SEC-002: Search latency high. **Updated Data Freshness Monitoring**: For Tier-1 corridors, alarms should trigger if the age of the latest Silver quote exceeds 15 minutes for more than 5% of checks or any corridor exceeds 20 minutes. For cold corridors, alarms trigger if data is older than 4 hours. Ensure dashboards and alerts reflect these new thresholds.			

Fix-It Plan and Readiness Updates

1. Performance SLAs and "Fast" Definition

- **No UI Waiting for Scrapes:** Clarify that the frontend never waits on a live provider scrape during a user search. All searches should hit our database/cache, ensuring instant responses.
- **Search Latency Target:** Define "fast" as a P95 latency under 200 ms for search API responses. In other words, 95% of search queries should return provider results from Redis/Postgres in under 0.2 seconds.
- **Data Freshness SLAs:** Hot (Tier-1) corridors must have data not older than 15 minutes, and cold (Tier-3) corridors up to 4 hours old at most. Adjust internal SLOs to align with the 15 min SLA for top corridors.

2. "Pulse" Feature Specification (Opportunity Ticker)

- **Feature Definition:** Describe Pulse as a real-time ticker highlighting corridors with unusually good deals. For example: Pulse surfaces corridors where the transfer rate is significantly better than its recent average.
- **Logic and Query:** Document the exact query used to identify opportunities. For example:

```
SELECT * FROM quotes
WHERE current_price < 0.95 * average_price_last_7_days;
```

In plain language, this flags any corridor quote that is >5% cheaper than the 7-day average for that route.

- **User Value:** Show the user deals where the transfer fee is 5% lower than usual right now. This ties the technical query back to a real user story.

3. Documenting "Sad Path" User Flows (Error & Edge Cases)

- **Password Reset Flow:** User clicks "Forgot Password" → Supabase triggers a reset email → User lands on `/update-password` form → User sets a new password → The system updates credentials and refreshes the session.
- **Email Verification Gate:** New users must verify their email before accessing certain features. Unverified users can search but cannot create alerts or perform costly actions until verification.
- **Unsubscribe One-Click Link:** Every notification email should include a one-click `unsubscribe_token` link. Clicking this link calls a public API endpoint (no login

required) which sets the user's notification preference to inactive (e.g., `is_active = false` in the `notification_pref` table).

4. **Data Fetching Strategy and Frequency**

- **Tiered Schedule:**
  - **Tier 1 (Hot corridors):** High-volume or highly volatile routes (e.g., USD→INR, USD→MXN, GBP→NGN) fetched every 15–30 minutes during peak hours (8 AM–8 PM local business hours).
  - **Tier 2 (Warm corridors):** Medium-traffic routes (e.g., EUR→PHP, CAD→CNY) fetched approximately every 2 hours (cron).
  - **Tier 3 (Cold corridors):** Low-traffic or very stable routes (e.g., ZAR→BRL, JPY→VND) fetched once daily (overnight) and otherwise on demand if a user searches that corridor.
- **FX Rate Trigger (On-Demand Fetches):** Monitor an external mid-market FX rate feed; if the currency moves by >0.5% within 10 minutes, immediately trigger scrapes for all Tier-1 corridors involving that currency. Rationale: providers adjust consumer rates only when forex markets move, so unnecessary scrapes can be avoided.

5. **Scalability and Resilience Updates (Circuit Breakers)**

- **Stale Data Fallback:** If a provider's scraper fails or data is outdated, serve the last known quote with a flag indicating it's stale. The UI should display a warning (e.g., "Data from 2 hours ago"). Cap the age of fallback data (e.g., do not serve data older than 24 hours).
- **Decoupled Scraper Queue (SQS + Lambdas):** Scraping tasks run asynchronously via a queue. The scheduler enqueues messages for each provider/corridor into a queue (e.g., `CollectionTaskQueue`). A fleet of headless collector Lambdas processes tasks, performs scrapes using Playwright/Puppeteer, and writes results to the database. The API never waits on scrapes; failed tasks can be retried or moved to a Dead Letter Queue.

6. **Build-vs-Buy Decisions (Tooling Choices)**

- **Auth:** Use Supabase Auth for user management (sign-up, login, password resets, email verification). No custom auth system will be built.
- **Payments:** Use Stripe Checkout for any payment or subscription upgrades. Integrate Stripe's hosted checkout and webhook callbacks; do not implement custom credit-card forms or payment processing logic.
- **Transactional Email:** Use AWS SES (or Postmark) for sending verification emails, alerts, etc. Leverage SES features for unsubscribe and bounce handling; do not use ad-hoc SMTP.

- **Proxy/Scraping Infrastructure:** Use commercial proxy services (e.g., Bright Data or Smartproxy) for IP rotation. Do not build our own IP rotation network.
- Record these decisions in the ADR/Tech Choices page with reasoning (e.g., choose proven third-party services to save time and ensure reliability/security).

7. Final Readiness Checklist Updates

- **Schema Locked-In:** Finalize and document the database schema for core tables (Providers, Corridors, Quotes, Users). Include any fields or indexes needed for Pulse (e.g., index on `corridor_id, created_at` to compute 7-day averages; consider materialized views).
- **Scraping Strategy Scripted:** Prepare a configuration file (e.g., `corridors_schedule.json`) listing all corridors by tier (Tier 1, 2, 3) according to the Smart-Tier schedule.
- **Walking Skeleton Deployed:** Deploy a minimal "Hello World" pipeline: fetch one corridor's quote (e.g., USD→INR from one provider), store it in the DB, and ensure the API can return it. Use this as the initial vertical slice.
- **Readiness Confirmation:** After updating documentation with all the above changes, ensure that all ambiguous points are clarified and prerequisites are met. When each checklist item is checked (schema done, strategy in place, skeleton running), the project can move from planning to coding.

With these updates in our Confluence documentation, the team and stakeholders will have a shared understanding of the plan, enabling a smooth transition from the audit findings to concrete engineering specifications.

🗒️ ADR Index

Purpose: Chronological index of architecture decisions (date, status, rationale) with links to full ADRs.

ADR ID & Title	Date	Status	Summary (Rationale)
ADR-2023-001: 3-Plane Architecture for Data Separation	December 2025	Accepted	Adopt Plane A (delivery), Plane B (ingestion), Plane C (publishing) to enforce hard separation between raw and derived data. Ensures compliance by construction; protects Golden Rule.
ADR-2023-002: Bronze–Silver–Gold Layering	December 2025	Accepted	Bronze = immutable audit ledger; Silver = operational/transactional window (prunable); Gold = derived analytics + external deliverables under strict controls.
ADR-2024-001: Gold Publisher Protocol (N≥3 Rule)	December 2025	Accepted	External outputs must pass publisher gates (≥3 providers, dominance checks, time bucketing, rounding/suppression) to prevent reverse engineering and comply with

			licensing/confidentiality.
ADR-2024-002: No Evasion / Stop-on-Block Policy	December 2025	Accepted	Ethical collection policy: no login scraping, no CAPTCHA solving, no ToS bypass, stop on any block signal. Circuit breakers + rights matrix enforcement require human review when posture changes.

🛂 Admin Dashboard (Ops Console)

Purpose

The Admin Dashboard is Remit-Scout's internal single pane of glass for operating the platform safely at scale. It centralizes configuration, enforcement, and monitoring across:

- Compliance & rights
- Ingestion operations (Plane B)
- Publishing gates / quarantines (Plane C)
- Monetization (sponsored placements, affiliate links)
- Analytics, pixels, and privacy/consent
- Identity & entitlements (Supabase + Stripe)
- Operations, incident controls, and audit/evidence

Designed for:

- Future engineers: fast onboarding + consistent mental model
- Auditors/investors: explicit boundaries + evidence trails
- LLM automations: least-privilege, safe, and auditable actions

---

Guardrails (Non-Negotiables)

These must always hold, regardless of feature pressure:

1. Golden Rule
   🚨 Raw artifacts never leave Plane B.
   The console must not expose Bronze/raw data to Plane A, end users, or third parties.
2. Rights First
   The Data Rights Matrix is authoritative. No admin action (including sponsorship/marketing) can bypass:

- Allowed_Collect
- Allowed_B2C
- Allowed_B2B
- Stoplist_Status (Active / Paused / Legal Hold)

3. Withhold > Publish
   If there is ambiguity, default to withhold (quarantine / internal-only / pause) rather than publish or expand access.
4. Stop-on-Block (Proxy-aware)
   Residential proxies are allowed for Plane B ingestion for localization/reliability, but explicit block signals (CAPTCHA/403/429/anti-bot messaging) trigger immediate stop + escalation. No retry-around-blocks.
5. Ads Never Re-Rank
   Sponsored placements are clearly labeled and never change organic ranking logic or comparison results.
6. Privacy & Consent
   No non-essential tracking scripts (marketing pixels, heatmaps, etc.) load before consent where required. Provide a kill-switch for marketing/analytics.

Where It Lives (3-Plane Model)

- Plane A (Delivery Surface): the Admin Dashboard is a Plane A internal application + API.
  - May read/write approved configuration and policy tables in Silver (Aurora Postgres).
  - May trigger workflows consumed by Plane B/C workers.
  - Must never fetch/stream Bronze/raw artifacts to the UI.
- Plane B and Plane C remain executors.
  - Admin sets policy.
  - The planes enforce.

Environment separation:

- Dev / Staging / Prod dashboards are isolated.
- No cross-environment keys or sessions.
- Recommended: separate domains + CloudFront/WAF + optional staff IP allowlist for prod.

Primary Users & Roles (RBAC)

All access is role-based, least-privilege, MFA-protected, and fully auditable.
Enforcement is server-side (never “security by UI”).

Suggested roles:

- Platform Admin: full access; emergency actions; owns RBAC and audit exports

- Compliance Admin: rights matrix, stoplists/legal holds, approvals, audit evidence
- Ops / On-call: ingestion controls, budgets, circuit breakers, incidents, health
- Data Product Admin: publisher quarantine review, methodology/version mgmt, restatements/backfills
- Growth / Marketing Admin: sponsored placements + affiliate templates + disclosures (no rights/stoplists)
- Support Admin (optional): support utilities (entitlements lookup, unsubscribe fixes); no ingestion/publishing control

Capability matrix (example):

- Rights Matrix edit → Platform Admin, Compliance Admin
- Stoplist/Legal Hold → Platform Admin, Compliance Admin
- Ingestion pause/resume → Platform Admin, Ops/On-call
- Circuit breaker open/close → Platform Admin, Ops/On-call
- Publisher quarantine/review → Platform Admin, Data Product Admin
- Sponsored placements → Platform Admin, Growth/Marketing Admin
- Analytics/pixels registry → Platform Admin, Compliance Admin, Growth/Marketing Admin
- Consent/cookies config → Platform Admin, Compliance Admin
- Identity view (read-only) → Platform Admin, Support Admin
- Audit export → Platform Admin, Compliance Admin

Authentication & Gatekeeping

- Auth: Supabase JWT verification (server-side).
- Authorization: explicit RBAC (server-side).
- Network: admin domain behind CloudFront/WAF + optional staff IP allowlist.
- Sessions: short-lived tokens; secure cookies; SameSite; CSRF protection if cookie-based sessions are used.
- Logging: every admin mutation emits an immutable audit event.

What the Console Manages (Scope)

The console manages configuration and decisions — not raw artifacts.

A) Provider & Rights Management (Compliance-first)

What admins can do:

- View/edit the Data Rights Matrix (provider permissions + constraints)
- Set Stoplist_Status (Active / Paused / Legal Hold)
- Attach reason + evidence links for any change
- View full change history (who/when/why)

Hard rules:

- A provider cannot go live unless present in the matrix.
- Sponsorship cannot override Allowed_B2C or display constraints.
- Rights downgrade triggers default stop:
  - pause collectors (Plane B)
  - block publishing and/or quarantine outputs (Plane C)

B) Ingestion Ops Console (Plane B controls)

What admins can do:

- View provider health (success/blocked/error rates)
- Pause/resume collection
- Open/close circuit breaker (reason required)
- Adjust budgets/frequency within approved limits
- Manage proxy posture (policy-level configuration only)

Proxy posture (new policy):

- Allowed: localization + reliability + controlled distribution
- Forbidden: bypass explicit stop signals; “IP cycling” to evade blocks
- Required:
  - log proxy country/region per run (for audit)
  - store block evidence metadata (timestamps, response codes, run_id) WITHOUT exposing raw pages in the console

C) Gold Publisher Ops (Plane C controls)

What admins can do:

- View publish runs and quarantine items
- Inspect failing gates (no raw artifacts; show only derived metadata)
- Approve internal-only outputs (if policy allows)
- Launch restatements/backfills (append-only, PIT-safe)
- Maintain methodology_version and algorithm_version attribution per publish run

Override policy:

- Overrides default to internal-only unless Compliance approves external release.
- Every override logs:
  - approver
  - reason
  - which gate was overridden
  - scope + expiry
  - backfill/restatement plan (if needed)

D) Sponsored Content & Affiliate Admin (Curated / First-Party)

Sponsored placements:

- Created only by authorized admins (manual curation; no external ad network)
- Always clearly labeled as “Sponsored”
- Must never change organic ranking logic or comparison results
- Allowed only if provider is Allowed_B2C = true (and all display constraints still apply)

Affiliate links:

- Manage affiliate link templates (destination allowlists, validated params)
- Ensure disclosures are present and accurate
- Maintain a “safe destination list” (prevent arbitrary redirects)

E) Analytics / Pixels / Heatmaps Admin (Privacy governance)

Tool registry must include:

- Tool name (e.g., GA4, Meta Pixel, heatmap vendor)
- Purpose (why it exists)
- High-level data captured (no raw PII)
- Event schema list
- Retention expectations
- Consent category required (e.g., Necessary / Analytics / Marketing)

Capabilities:

- Enable/disable tools via config (kill switches)
- Enforce consent gating (load only after consent where required)
- “Strict mode” (EU / privacy-first mode): disable non-essential tracking by default

Hard rules:

- No non-essential scripts without consent where required.
- Event schemas are documented and change-controlled.
- No raw PII in analytics events.

F) Identity & Entitlements Admin (Supabase + Stripe)

Defaults:

- Read-only view of users/plans/entitlements/quotas
- View auth anomalies (rate limit hits, unusual failures, suspected abuse)

Limited writes (policy-gated; logged; evidence required):

- Revoke refresh tokens (incident response)
- Disable account (policy & support workflow)
- Correct marketing opt-in state (with evidence)

Hard rules:

- No “impersonate user” by default.
- Supabase service role key never exposed client-side.
- All admin actions must be audited.

G) Audit Log & Evidence Center

Must provide:

- Search audit by actor/role/action/entity/time
- Before/after diffs for changes
- Reason + evidence links
- Export for compliance review

Audit event should include:

- event_id
- actor_id
- actor_role
- action (verb.noun)
- entity_type + entity_id
- reason (required for mutations)
- evidence_links (optional)

- before_snapshot (structured)
- after_snapshot (structured)
- timestamp

Data Storage & Retention (Silver / Aurora Postgres)

Store configuration in Silver (never Bronze/raw artifacts).

Suggested tables (illustrative):

- admin_user_role
- admin_audit_event (append-only)
- provider_rights (Data Rights Matrix)
- provider_stoplist_history (append-only)
- collector_budget_config
- collector_circuit_state
- proxy_policy_config
- sponsored_placement
- affiliate_link_template
- tracking_tool_config
- consent_config / cookie_banner_config
- feature_flag

Retention (tune as needed):

- Audit logs: 1–3+ years
- Ops run summaries: 90–180 days
- Rights matrix history: append-only; retain indefinitely or per legal policy

---

Admin API Surface (Conceptual)

All UI actions go through server-side APIs. No direct DB writes from the browser.

Recommended namespaces:

- /api/admin/providers/*
- /api/admin/rights-matrix/*
- /api/admin/ingestion/*
- /api/admin/publisher/*

- /api/admin/sponsored/*
- /api/admin/affiliate/*
- /api/admin/tracking/*
- /api/admin/consent/*
- /api/admin/audit/*

Every mutation must:

- enforce RBAC server-side
- require a reason string
- optionally accept evidence links
- emit an admin_audit_event
- support idempotency for safe retries (recommended)

Emergency Actions (On-call “Big Red Buttons”)

Every emergency action requires:

- elevated role
- explicit confirmation text
- an audit record
- an ops notification to the on-call channel

Examples:

- Pause provider → Stoplist_Status = Paused
- Legal hold provider → Stoplist_Status = Legal Hold
- Disable publishing for a dataset family → force quarantine/internal-only
- Disable all marketing tracking → privacy kill switch

Monitoring & SLOs (Minimum)

Track:

- Admin API availability + latency
- Admin error rates (especially mutations)
- Audit log write success (must be effectively 100%)
- Config propagation lag (admin change → enforcement)
- Suspicious admin activity (anomaly detection)

Suggested targets (tune per environment):

- Availability: ≥ 99.9% monthly
- p95 mutation latency: ≤ 500ms
- Audit write success: alert on any drop
- Config propagation lag: ≤ 60s

Boundary & Security Tests (Must Exist)

Critical boundaries:

- Admin UI/API cannot access Bronze/raw artifacts
- External publishing cannot bypass Gold Publisher gates
- Sponsorship cannot bypass Allowed_B2C or change organic ordering
- Stop-on-block triggers even when proxies are enabled
- Non-essential tracking scripts are blocked until consent is true

Acceptance test checklist (example):

- BND-001: No Bronze in Admin (deny + no data path)
- BND-002: Publisher gates enforced (fail → quarantine with reason)
- BND-003: Sponsorship cannot re-rank (organic order unchanged; sponsored labeled)
- BND-004: Stop-on-block honored with proxies (circuit opens; no bypass)
- BND-005: Consent gating (pixels/heatmaps blocked until consent)

Change Control

Any change to the following requires:

- ADR
- boundary/regression tests
- rollout plan + monitoring

Change-controlled areas:

- Rights posture, Allowed_* logic, stoplist enforcement
- Publisher gates/thresholds/rounding rules
- Sponsored rendering rules and disclosure policy
- Analytics event schemas / consent categories / pixel enablement rules

---

Recommended Page Tree Under This Page (Optional)

Admin Dashboard (Ops Console)

- Provider & Rights Management
- Ingestion Ops (Budgets, Schedules, Proxies, Stoplists)
- Gold Publisher Ops (Quarantine, Approvals, Restatements)
- Sponsored Content & Affiliate Admin
- Analytics, Pixels, Heatmaps & Event Governance
- Cookies & Consent Admin
- Identity / Entitlements Admin (Supabase + Stripe)
- Audit Log & Evidence
- Runbooks & Emergency Actions

🏛️ Compliance & Governance

> Why this page exists
>
> This page is intentionally separate from implementation details. Auditors and investors must be able to review our rights posture, publishing rules, and "stop" behaviors independently from code, infra diagrams, and vendor choices.
>
> If anything here conflicts with an implementation, the implementation is wrong.

---

0) Non-Negotiables

0.1 The Golden Rule (Restated)

🚨 **Raw data never leaves Plane B.** Only **Derived Data ("Gold")** leaves via **Plane C**.

If there is ambiguity, we default to **withhold** rather than publish.

0.2 What we will NOT implement (ever)

We do not do circumvention/evasion behaviors. Specifically we do not:

- scrape behind logins, create accounts, or use authenticated sessions to access quotes
- accept clickwrap / ToS prompts via automation to obtain access
- solve CAPTCHAs (farms, bypasses, or "workarounds")
- run "cat-and-mouse" evasion loops (fingerprint spoofing, "undetected" drivers, header rotation to mimic humans, **No proxies for overriding explicit stop signals.** Proxies (including residential networks) may be used for **localization and reliability** in Plane B, but **must not be used to continue collection after a hard block** (CAPTCHA/login wall/403/429/explicit anti-bot messaging).
- ignore robots/explicit blocks if the site signals "stop"

Our alternative is: **"polite observation"** + strict budgets + circuit breakers + takedown compliance + partnership-first path.

---

1) Data Rights Governance (Provider × Permission Ledger)

1.1 Data Rights Matrix (Single Source of Truth)

Purpose

The Data Rights Matrix is the authoritative ledger for what we are allowed to:

- Collect (**Allowed_Collect**)
- Use for B2C display (**Allowed_B2C**, with display constraints)
- Use for B2B derived publishing (**Allowed_B2B**, with aggregation constraints)
- Run right now (**Stoplist_Status** / **Circuit_Status**)

Minimum required columns

- provider_id
- provider_name
- collection_method (Tier A API / Tier B Headless / other)
- Allowed_Collect (Y/N)
- Allowed_B2C (Y/N + display constraints)
- Allowed_B2B (Y/N + aggregation constraints)
- Stoplist_Status (Active / Paused / Legal Hold)
- Reason / Notes (links to ToS excerpts, legal correspondence, approvals)
- Last_Reviewed_At, Owner

Operating rules

- Any new provider must be added to the matrix **before** a collector goes live.
- Any change in ToS or legal instruction → **update matrix same day**.
- Stoplist_Status is **enforceable by automation** (the scheduler MUST honor it).

1.2 Stoplist, Circuit Breakers, and “Stop-on-Block”

We implement a fail-safe control plane:

- **Stoplist_Status** is the durable human/Legal-controlled switch (Active / Paused / Legal Hold).
- **Circuit_Status** is the automatic safety switch (Open / Half-Open / Closed) driven by block signals and error thresholds.

Stop-on-block policy (hard rule)

If we detect block signals (captcha pages, 403/429 patterns, explicit anti-bot messaging, login walls, clickwrap gating):

- Immediately stop collection attempts for that provider
- Open circuit breaker and set Stoplist_Status = “Paused” (or “Legal Hold” if instructed)
- Notify Engineering + log evidence (timestamps, run_id, response metadata)

---

2) Derived Data Boundary (Publishing Governance)

Goal

Anything leaving Plane C (Gold export datasets, B2B shares, external files) must be:

- Derived
- Irreversible
- Rights-compliant
- Auditable

2.1 The “Gold Publisher” Protocol (The Publishing Gate)

Inputs

Candidate output (e.g., corridor index for day D):

- Derived from Silver/Growth staging
- Includes provenance metadata (counts, window bounds, algorithm version)

Hard gates (must pass)

A) Rights check

- Every contributing provider must have Allowed_B2B == true
- Dataset type must be allowed for that provider (respect provider-specific constraints)

B) N≥3 rule (minimum contributors)

- Must have ≥ 3 distinct independent providers for each published point
- If N < 3 → do not publish (or publish as internal-only with explicit flag)

C) Dominance checks (anti-reverse-engineering)

Prevent a single provider from effectively determining the result:

- Max weight per provider ≤ W_max (e.g., 50%)
- Top-2 combined weight ≤ W_top2 (e.g., 75%)

If dominance fails → withhold or broaden aggregation window until it passes.

D) Staleness / coverage

- Contributors must be within freshness bounds for the reference window

- If data coverage is too sparse → withhold and log reason

E) Precision reduction

To reduce reverse engineering risk:

- Apply rounding / quantization (e.g., basis-point rounding or fixed decimals)
- Avoid raw-like precision for small samples

F) Anomaly guardrails

- Reject candidates with extreme deltas vs trailing baseline unless explicitly approved
- Record reason + require human review for overrides

Outputs (auditing)

If published:

- Write to Gold “published” tables (append-only, PIT compliant)
- Emit an audit record with:
  - contributors_count
  - contributor_set_hash (or reference list stored internally)
  - algorithm_version
  - window_start / window_end
  - publisher_run_id

If withheld:

- Write to “quarantine” table with failure reason

2.2 Change control for Publisher gates

Any change to Publisher gates requires:

- ADR
- Legal/compliance review (if reversibility posture changes)
- Backfill/restatement plan (what changes, when, and how we communicate)

---

3) Collection Conduct (Proxy & Scraping Policy)

Intent

- We may use localization and proxy networks (including residential) to observe the correct public experience and maintain reliability.
- We do not use proxies to bypass explicit stop signals or violate restrictions.

Allowed Actions

- **Localization:** Choose egress country/region to match corridor/user locale being observed.
- **Reliability:** Distribute load and reduce accidental blocking within approved budgets.
- **Residential proxies permitted:** Approved residential proxy services may be used for Plane B collection.
- **Transparency:** Log proxy provider + egress country/region per run (audit).

Forbidden Actions (Non-Negotiable)

- No login scraping (no accounts, no authenticated sessions).
- No CAPTCHA solving farms or bypass techniques.
- No “cat-and-mouse” evasion loops (fingerprint spoofing, undetected drivers, etc.).
- **No proxy cycling after a hard block** (do not “try another IP” when stop signals appear).

Stop-on-Block Policy (keep, but strengthen)
Add one extra bullet:

- If blocked **even while using a proxy**, we still **stop immediately** and escalate; proxies do not grant override authority.

Add new ADR entry: `ADR-2025-001: Controlled Use of Residential Proxies`

- Decision: Residential proxies allowed for Plane B localization/reliability
- Non-negotiable: Stop-on-Block still enforced
- Auditing: proxy metadata logged per run
- Security: proxy creds stored in Secrets Manager; least-privileged access
- Risk notes: vendor review, allowlist endpoints, budget caps

Update/Supersede old ADR (your “No Evasion/Stop-on-Block Policy” ADR) with an amendment:

- “No-evasion” still stands; proxies are now a **permitted tool** under strict constraints.

---

4) Legal Escalations Runbook (Cease & Desist / Takedown)

Trigger events

- C&D received from a provider or legal representative
- Provider demands removal/stop of collection
- Material ToS change affecting Allowed_Collect / Allowed_B2B

Immediate actions (kill switch)

- Set provider Stoplist_Status = Legal Hold
- Disable collector scheduling for that provider (all environments if required)
- Disable any publishing that could reference that provider (if applicable)
- Preserve evidence:
  - last successful runs
  - block/response metadata
  - rights matrix state at time of incident
- Notify: Legal owner + Eng lead + On-call

Follow-up actions

- Confirm what datasets included the provider and whether any external deliverables need remediation
- Document outcome + decision in ADR / incident report
- Update Data Rights Matrix and any compliance documentation

What NOT to do

- Do not attempt to “test” access repeatedly after a legal request
- Do not alter collection method to regain access (no evasive changes)

---

5) The Missing Layer: Product Backbone Services (Plane A)

Why this matters

Your prior design had the right “Truth Engine + Lakehouse + Derived Boundary.” What was missing is the **Product Backbone** that turns the frontend into a real app.

These modules are **not** “nice-to-haves”—they are exactly what the frontend references today.

Plane A responsibilities (governance framing)

- Provides stable user-facing APIs and UX features
- Stores user data (PII) with strict access controls and retention
- Enforces entitlements server-side (frontend gating is UX only)
- MUST NOT have credentials/network paths to Bronze (raw artifacts)

5.1 Identity + Billing + Entitlements Service

Components

- Supabase auth (JWT verification; cache JWKs)
- Stripe checkout + webhooks
- Plan enforcement + quotas (Free/Plus/B2B)
- **/api/me** as the single source of truth

Compliance controls

- Server-side enforcement on every request (no client-trusted entitlements)
- Idempotent webhook handling and audit logs for plan changes
- Minimal local billing “truth cache” (user_plan) to reduce reliance on third parties during incidents

5.2 Watchlist + Alerts + Notifications Service

Components

- Persist watchlists & alerts
- Alert evaluation workers (realtime/hourly/daily)
- Alert history, notification prefs, unsub handling
- Guest alert support (double opt-in)

Compliance controls

- Alerts must evaluate only from permitted sources (B2C latest views / Gold aggregates / FX service)
- Anti-spam and consent controls: cooldowns, unsub, suppression lists
- No provider-sensitive naming in outbound alerts unless explicitly permitted

5.3 History + Exports Service

Components

- Real time-series backed by Gold (ClickHouse)
- Export job queue (CSV/PDF) to S3 + signed downloads

Compliance controls

- Exportable datasets originate from Gold-export tables only
- Plan-gated history ranges (e.g., 30d free vs 365d plus)
- Export jobs are async + auditable; files are time-limited and access-logged

5.4 FX Mid-Market Rate Service

Components

- Ingest mid-market feed (licensed provider)
- Store aligned rates for QuoteRecord computation + charting

Compliance controls

- Only use licensed/defensible FX source for mid-market alignment
- Track provenance (source, timestamp, freshness SLOs)

5.5 Telemetry Ingestion Service

Components

- Persist recent searches, outbound clicks
- Feed “Intent” indices (with privacy gating)

Compliance controls

- Data minimization: store anon_session_id (random), not IP
- Rule-of-5 gating for any published intent index
- Retention policy: keep “hot window” operationally; archive or aggregate for long-term

5.6 Contact + Newsletter Intake

Components

- Store submissions
- Double opt-in for marketing/newsletter lists
- Optional push to ESP later (Mailchimp/etc)

Compliance controls

- Explicit consent (double opt-in) + easy unsubscribe
- Separate marketing consent from product alerts

---

6) Crosswalk: Frontend Gaps → Backend Deliverables

A) Persist watchlists/alerts (today local)

Frontend: useWatchlist.ts, useAlerts.ts, tracking.ts WatchTarget types
Backend: Watchlist + Alerts Service (Aurora Postgres + APIs)

Deliverables

- watchlist_item table supporting all WatchTarget types via target_type + target_payload_jsonb
- /api/watchlist CRUD

- /api/alerts CRUD (linked to watchlist items)

B) Alert evaluation + notification pipeline + prefs

Frontend: dashboard.vue, SaveAlertModal.vue
Backend: Alert Evaluator + Notification Worker (EventBridge → SQS → Lambda/ECS + SES)

Deliverables

- evaluation scheduler for realtime/hourly/daily
- alert_event history
- notification_prefs + unsubscribe support
- bounce/complaint suppression list (SES feedback)

C) Guest alerts (email-only form)

Frontend: RateAlertForm.vue
Backend: Guest subsystem (separate tables + double opt-in)

Deliverables

- guest_subscriber (pending→active→unsubscribed)
- /api/guest/alerts create (returns “check email to confirm”)
- /api/guest/confirm and /api/guest/unsubscribe

D) Replace local entitlements/auth with real sessions + Stripe

Frontend: useAuth.ts, useEntitlements.ts, checkout.vue, success.vue
Backend: Identity/Billing/Entitlements Service

Deliverables

- /api/me returns plan + entitlements + quotas
- /api/billing/checkout-session creates Stripe checkout
- /api/billing/webhook processes subscription lifecycle
- /api/billing/portal optional (customer portal link)

E) Dashboard history/export tabs with real time-series and export jobs

Frontend: dashboard.vue tabs
Backend: History + Exports Service (ClickHouse + S3 exports)

Deliverables

- /api/history/corridor backed by Gold tables

- /api/exports create job + status + signed download
- plan gating + quotas enforced

F) Replace mocks with QuoteRecord/Gold + add FX feed + telemetry + contact/newsletter

Frontend: providers.get.ts, bank-vs-specialist.get.ts, pulseMockApi.ts, recent-searches.post.ts, click.post.ts, contact.vue, NewsletterSignup.vue
Backend: Quote API + Pulse API + FX Service + Telemetry + Contact

Deliverables

- /api/quotes/current from Aurora latest_quote_by_provider
- /api/pulse/overview, /api/pulse/chart/:id from ClickHouse Gold
- /api/fx/midmarket internal + ingestion job
- /api/telemetry/recent-search and /api/telemetry/click
- /api/contact and /api/newsletter/subscribe with double opt-in

---

7) Concrete Aurora Postgres Data Model (Operational)

7.1 Watchlists: support every WatchTarget type without migrations

watchlist_item

- id (uuid, pk)
- owner_type enum('user','guest')
- user_id uuid null
- guest_id uuid null
- target_type text (corridor | provider | pulse_metric | custom | ...)
- target_payload jsonb (corridor_id, amount_bucket, payin/payout, provider_id, chart_id, etc.)
- label text null
- created_at, updated_at
- deleted_at null

Constraint: exactly one of (user_id, guest_id) must be non-null.

Indexing:

- (user_id, created_at desc)
- GIN index on target_payload if needed

7.2 Alerts: definition + state + history

alert_rule

- id uuid pk
- watchlist_item_id uuid fk
- metric enum('recipient_gets','rci_bps','all_in_cost_send','mid_market_rate','leader_edge_bps', ...)
- comparator enum('gte','lte','gt','lt','crosses_above','crosses_below')
- threshold numeric
- frequency enum('realtime','hourly','daily')
- cooldown_minutes int default 360
- enabled bool
- created_at, updated_at

alert_state

- alert_id uuid pk fk
- last_evaluated_at
- last_value numeric null
- in_alarm bool default false
- last_triggered_at
- last_notified_at
- snoozed_until null
- version int (safe concurrent updates)

alert_event

- id uuid pk
- alert_id uuid
- triggered_at timestamptz
- value numeric
- message text
- context jsonb (corridor_id, bucket, chart_id, etc.)
- notification_status enum('queued','sent','failed')
- provider_safe bool (true unless partner-permitted provider naming)

7.3 Notification prefs + suppression

notification_pref

- owner_type enum('user','guest')
- user_id uuid null
- guest_id uuid null
- channel enum('email')
- timezone text default 'UTC'
- daily_send_hour int default 9
- digest_enabled bool default true
- marketing_opt_in bool default false
- unsubscribed bool default false
- created_at, updated_at

email_suppression

- email_hash text pk
- reason enum('bounce','complaint','manual')
- created_at

7.4 Guest alerts (double opt-in; isolated subsystem)

guest_subscriber

- id uuid pk
- email citext unique
- status enum('pending','active','unsubscribed')
- verify_token_hash text
- unsubscribe_token_hash text
- created_at, verified_at, unsubscribed_at
- source text (RateAlertForm, ...)

7.5 Entitlements + Stripe cache

user_plan

- user_id uuid pk
- plan_code enum('free','plus','enterprise')
- status enum('active','past_due','canceled','trialing')
- stripe_customer_id text

- stripe_subscription_id text
- current_period_end timestamptz
- cancel_at_period_end bool
- created_at, updated_at

plan_usage_counters (recommended)

- user_id uuid pk
- period_start, period_end
- alerts_count int
- exports_count int
- api_calls_count int
- updated_at

7.6 Exports

export_job

- id uuid pk
- user_id uuid
- job_type enum('history_csv','history_pdf','pulse_csv')
- params jsonb
- status enum('queued','running','done','failed')
- s3_key text null
- expires_at timestamptz
- created_at, started_at, finished_at
- error text null

7.7 Telemetry (privacy-safe)

telemetry_search_event

- id uuid pk
- ts timestamptz
- anon_session_id text null
- user_id uuid null
- corridor_id text
- amount_bucket int
- payin text, payout text

- utm jsonb null
- page_path text null

telemetry_outbound_click

- id uuid pk
- ts timestamptz
- anon_session_id text null
- user_id uuid null
- provider_id text
- corridor_id text
- target_url text (sanitized/normalized)
- page_path text
- utm jsonb null

7.8 Contact + newsletter

contact_submission

- id uuid pk
- email citext
- name text null
- message text
- meta jsonb (utm, page)
- created_at

newsletter_subscriber

- id uuid pk
- email citext unique
- status enum('pending','active','unsubscribed')
- verify_token_hash
- unsubscribe_token_hash
- created_at, verified_at, unsubscribed_at
- source text

---

8) Alert Evaluation & Notification Pipeline (realtime/hourly/daily)

8.1 Evaluation sources (what values do we check?)

Alert metrics are computed from ONE of:

- B2C current quotes (Aurora latest_quote_by_provider / “best quote view”)
- Gold aggregates (ClickHouse gold_export.cdp_4h / gold_export.cdp_daily)
- Mid-market rate (FX service)

This prevents alerts from requiring raw internal quote streams.

8.2 Scheduling (simple, scalable, cost-controlled)

- Use SQS to queue evaluation tasks; workers (Lambda or ECS) execute.

Realtime (Plus)

- every 5 minutes for corridors with active realtime alerts (dedupe/cooldown makes this safe)

Hourly

- EventBridge hourly: enqueue tasks per corridor with hourly alerts

Daily (Free + guest)

- EventBridge daily at user preferred hour (bucket by hour): enqueue tasks for those users/guests

8.3 Worker logic (must prevent spam)

For each alert:

- fetch current metric value (from Gold or latest quote view)
- evaluate comparator (crosses_above/below uses last_value)
- apply cooldown/hysteresis
- write alert_event + update alert_state
- enqueue email send (don’t block evaluation on SES)

8.4 Before sending any email

- check notification_pref.unsubscribed
- check email_suppression (bounces/complaints)
- check guest status == active
- log send attempt + SES message id

---

9) Implementation Order (to unblock frontend fast)

Week 1 (frontend unblockers)

- /api/me + JWT verification + basic user_plan cache (free only)
- /api/watchlist + /api/alerts persistence (no evaluation yet)
- Replace /api/quotes/current mock with Aurora latest table
- Telemetry endpoints (recent-search, click) writing to Aurora

Week 2

- Daily alert evaluation + SES notifications + unsubscribe links
- ClickHouse Gold history endpoints for dashboard charts

Week 3

- Stripe checkout + webhooks → Plus plan enforcement
- Export jobs (CSV first, then PDF)

Week 4

- Hourly/realtime alert frequencies for Plus
- Pulse endpoints fully backed by ClickHouse + MEL events

---

10) References / Related Pages

- 🏗️ System Architecture (The Machine)
- 📊 Methodologies (RCI/CDP/MEL definitions + suppression rules)
- 💾 Data Models & Schemas (full DDL and ClickHouse table definitions)
- 🛠️ Operations & Runbooks (on-call, backfills, stoplist ops, Stripe ops, SES ops)

📌 Planning & Documentation → Plans & Tasks (Playbook)

Purpose

Reduce technical debt by making Confluence the source of truth, and generating consistent epics/tasks (with acceptance criteria) directly from structured docs.

A) Recommended Confluence Page Map (RSE space)

1) 🏛️ Compliance & Governance (Shield)

What goes here:

- non-negotiables (no circumvention, stop-on-block)
- rights matrix policy + operating rules
- Gold publisher gates + change control
- legal incident response runbooks (C&D)
- product backbone compliance framing (PII, consent, exports, entitlements)

2) 🏗️ System Architecture (Machine)

Subpages (suggested):

- Plane A: Product Backbone Services (APIs, bounded contexts, dependencies)
- Plane B: Truth Engine (scheduler, collectors, normalizer, stoplist enforcement)
- Plane C: Gold Products (CDP/MEL/indices, publisher gate, exports)
- Cross-plane boundaries (IAM/network/credentials enforcement)
- End-to-end data flow diagrams

3) 📊 Methodologies

Subpages:

- QuoteRecord definition & calculations (recipient_gets, fees, RCI inputs)
- CDP/MEL definitions and suppression rules
- dominance rules & rounding policy
- mid-market alignment methodology

4) 💾 Data Models & Schemas

Subpages:

- Aurora schema (operational tables; migrations; constraints)
- ClickHouse schema (gold_internal vs gold_export)
- S3 partitions/retention (Bronze + exports + debug TTL)

5) 🛠️ Operations & Runbooks

Subpages:

- on-call + incident response (engineering side)
- stoplist/circuit breaker operations
- backfills/restatements
- Stripe ops (webhooks, plan reconciliation)
- SES ops (bounces/complaints, suppression)

- data freshness SLOs + dashboards

B) Doc Template (use this for every major module/page)

Use the same sections so LLM outputs are consistent:

- Overview (what it does / why it exists)
- Non-goals
- Invariants / compliance constraints
- Interfaces (endpoints, events, schedules)
- Data model (tables + key constraints)
- Failure modes & safe defaults
- Observability (metrics, alarms, dashboards)
- Rollout plan (flags, staging, backfill)
- Ownership & review cadence
- Links (ADR, runbooks, tickets)

C) Turning a doc section into Epics/Stories (LLM workflow)

Step 1 — Normalize inputs

For a given feature/module, ensure the doc explicitly lists:

- user-facing behaviors (what the frontend expects)
- backend deliverables (endpoints, tables, jobs)
- compliance gates (rights, derived boundary, consent)
- success metrics + failure behaviors

Step 2 — Generate backlog at three levels

1. Epic (outcome)
2. Stories (vertical slices delivering usable increments)
3. Tasks (schema, API, worker, tests, dashboards, runbooks)

Step 3 — Enforce a Definition of Done (DoD)

For each story:

- API contract documented
- migrations added + tested
- unit tests + integration tests

- logs/metrics/alarms added
- runbook written (how to operate / rollback)
- compliance checklist passed (rights, derived boundary, consent, retention)

D) Example: “Watchlists + Alerts” backlog generated from this doc

Epic: Watchlists + Alerts MVP (persist + daily evaluation)

Story 1: Persist watchlists (no evaluation)

- Create watchlist_item table + constraints/indexes
- Implement /api/watchlist CRUD + auth middleware
- Add server-side quota enforcement hooks (even if free-only initially)
- Add tests + minimal monitoring (errors/latency)
- Update docs + add runbook “Watchlist API operations”

Acceptance criteria:

- user can create/read/update/delete watchlist items across all WatchTarget types
- data survives refresh/login and is accessible from multiple devices

Story 2: Persist alert rules + state

- Create alert_rule, alert_state, alert_event tables
- Implement /api/alerts CRUD
- Add cooldown and crosses_above/below state handling

Acceptance criteria:

- alerts can be created/updated/disabled
- state updates are concurrency-safe (versioned updates)

Story 3: Daily evaluator + SES notifications

- EventBridge daily schedules → enqueue SQS tasks
- Worker evaluates from Gold or latest_quote_by_provider only
- Implement notification_pref + unsubscribe links
- Implement email_suppression ingestion (SES bounces/complaints)

Acceptance criteria:

- daily alerts send once per day max per rule, respect cooldown
- unsubscribed/suppressed emails are never sent

💾 Data Models & Schemas

What this page is (and isn't)

This page is the **canonical schema intent** for Remit-Scout's data stores. It defines:

- **What tables exist**, what they mean, and what invariants they must uphold
- **How Plane A product features map to operational tables**
- **How Gold (ClickHouse) is structured for PIT + export safety**
- **Which data is allowed to flow where** (boundary reminders)

This page is not a full infrastructure diagram. See 🏗️ *System Architecture* for topology and deployment details.

Non-negotiable constraints that affect schema design

These are governed in 🏛️ *Compliance & Governance*, but repeated here because they drive **what we store** and **what we refuse to store**:

- **No circumvention / evasion:** no login scraping, no clickwrap acceptance via automation, no CAPTCHA solving, no "cat-and-mouse" bot evasion loops.
- **Stop-on-block:** block signals immediately pause the provider and trip circuit breakers.
- **Derived-data boundary:** anything leaving Plane C must be **derived + irreversible + rights-compliant + auditable**.
- **Fail-safe defaults:** withhold over publish; stop over evade.

Layer model (Bronze / Silver / Gold)

We maintain three layers with different invariants:

Bronze (S3) — immutable ingestion ledger (audit + replay)

- Append-only objects (structured captures / raw-ish payloads depending on rights).
- Primarily for **provenance, replay, and audit**.
- **Not** directly queried by product APIs.

Silver (Aurora Postgres) — operational truth (product + latest quotes)

- Normalized QuoteRecord + ingestion metadata.

- Product Backbone state (users' watchlists/alerts, entitlements cache, exports, telemetry, contact/newsletter).
- Optimized for **low-latency reads + transactional correctness**.

Gold (ClickHouse) — analytical truth (PIT, restatements, B2B)

- Derived datasets (indices, panels, events, export-safe time series).
- Supports **PIT semantics** (reference_time vs knowledge_time).
- Split into:
  - **gold_internal** (internal-only, may be more granular)
  - **gold_export** (safe to publish/share; derived + suppressed + bucketed + rounded)

Naming + modeling conventions

Common columns

- `created_at`, `updated_at` on mutable operational tables.
- `deleted_at` for soft deletes where needed (watchlists, alerts).
- Prefer `timestamptz` for event timestamps.

IDs

- Use stable string IDs for externally meaningful concepts (`provider_id`, `corridor_id`).
- Use UUID for user-owned entities (`watchlist_item.id`, `alert_rule.id`, etc.).

JSONB usage (Aurora)

Use `jsonb` for polymorphic payloads and forward compatibility:

- `watchlist_item.target_payload`
- `alert_event.context`
- provider-specific fee breakdowns where schema differs across providers

Enums

Prefer enums for bounded states that power logic:

- `stoplist_status`, `circuit_status`, `plan_code`, `export_job.status`, etc.

---

1) Silver Layer — Aurora Postgres (Operational)

> NOTE: Field names below are canonical intent. Align exact names/types with migrations.

1.1 Ingestion + QuoteRecord domain (Plane B)

provider

column	type	notes
provider_id	text (PK)	canonical string (e.g., `wu`)
display_name	text	
status	enum	`active` / `paused` / `legal_hold`
rights_tier	enum	`tier_a` / `tier_b` / `other`
created_at	timestamptz	
updated_at	timestamptz	

corridor

column	type	notes
corridor_id	text (PK)	stable canonical ID
send_currency	text	ISO-4217 where possible
receive_currency	text	ISO-4217 where possible
send_country	text	optional
receive_country	text	optional

created_at	timestamptz	
updated_at	timestamptz	

ingestion_run

column	type	notes
run_id	uuid (PK)	
provider_id	text (FK)	→ provider.provider_id
collector_type	enum	`tier_a_api` / `tier_b_headless` / `other`
started_at	timestamptz	
finished_at	timestamptz	
status	enum	`success` / `blocked` / `error`
proxy_country	text	if applicable
error_code	text	sanitized
error_detail	text	sanitized
created_at	timestamptz	

quote_record (operational truth; “hot window”)

column	type	notes
quote_id	uuid (PK)	
provider_id	text (FK)	
corridor_id	text (FK)	or store currency pair fields
send_amount	numeric	

fee_amount	numeric	
fee_currency	text	optional if always send currency
receive_amount	numeric	
implied_fx_rate	numeric	receive per send (derived)
collected_at	timestamptz	reference time (observation)
ingested_at	timestamptz	knowledge time (when learned)
ingestion_run_id	uuid (FK)	
bronze_object_key	text	pointer to Bronze provenance
parser_version	text	
quality_flags	jsonb	stale/partial/layout change/etc
created_at	timestamptz	

Silver invariants

- Every `quote_record` must be traceable to Bronze via `bronze_object_key`.
- Silver may be operationally pruned; **Bronze is the long-term record**.
- No consumer endpoint should require direct Bronze reads.

latest_quote_by_provider (B2C fast path; maintained by upsert)

Grain: one row per (corridor_id, amount_bucket, payin/payout, provider_id)

column	type	notes
corridor_id	text	
amount_bucket	int	canonical buckets
payin	text	e.g. `bank` / `card`

payout	text	e.g. `cash` / `bank` / `mobile`
provider_id	text	
collected_at	timestamptz	
send_amount	numeric	
fee_amount	numeric	
receive_amount	numeric	
implied_fx_rate	numeric	
quality_flags	jsonb	
updated_at	timestamptz	

Indexes

- UNIQUE: `(corridor_id, amount_bucket, payin, payout, provider_id)`
- Read pattern index for quotes/current: `(corridor_id, amount_bucket, payin, payout)`

---

2) The missing layer — Product Backbone Services (Plane A)

These are first-class backend modules that turn the frontend into a real product. This section defines the **operational schemas** they require.

2.1 Identity + Billing + Entitlements

user_plan (billing truth cache)

column	type	notes
user_id	uuid (PK)	Supabase user id
plan_code	enum	`free` / `plus` / `enterprise`

status	enum	`active` / `past_due` / `canceled` / `trialing`
stripe_customer_id	text	
stripe_subscription_id	text	
current_period_end	timestamptz	
cancel_at_period_end	bool	
created_at	timestamptz	
updated_at	timestamptz	

plan_usage_counters (recommended)

column	type	notes
user_id	uuid (PK)	
period_start	timestamptz	
period_end	timestamptz	
alerts_count	int	
exports_count	int	
api_calls_count	int	optional
updated_at	timestamptz	

billing_webhook_event (idempotency + audit)

column	type	notes
id	uuid (PK)	
provider	enum	`stripe`
event_id	text	Stripe event id (unique)

received_at	timestamptz	
processed_at	timestamptz	null until processed
status	enum	`received` / `processed` / `failed`
error	text	nullable
payload	jsonb	optional redacted copy

2.2 Watchlists (supports all WatchTarget types without migrations)

watchlist_item

column	type	notes
id	uuid (PK)	
owner_type	enum	`user` / `guest`
user_id	uuid	nullable
guest_id	uuid	nullable → guest_subscriber.id
target_type	text	e.g. `corridor`, `provider`, `pulse_metric`, `custom`
target_payload	jsonb	corridor_id, amount_bucket, payin/payout, provider_id, chart_id…
label	text	nullable
created_at	timestamptz	
updated_at	timestamptz	

deleted_at	timestamptz	nullable

Constraints

- Exactly one of `(user_id, guest_id)` must be non-null.

Indexes

- `(user_id, created_at desc)`
- `(guest_id, created_at desc)`
- Optional: GIN on `target_payload` if querying nested keys is required

2.3 Alerts (definition + state + history)

alert_rule

column	type	notes
id	uuid (PK)	
watchlist_item_id	uuid (FK)	→ watchlist_item.id
metric	enum	`recipient_gets`, `rci_bps`, `all_in_cost_send`, `mid_market_rate`, `leader_edge_bps`, …
comparator	enum	`gte`, `lte`, `gt`, `lt`, `crosses_above`, `crosses_below`
threshold	numeric	

frequency	enum	`realtime`, `hourly`, `daily`
cooldown_minutes	int	default 360
enabled	bool	
created_at	timestamptz	
updated_at	timestamptz	

alert_state (mutable runtime state; concurrency-safe)

column	type	notes
alert_id	uuid (PK/FK)	→ alert_rule.id
last_evaluated_at	timestamptz	
last_value	numeric	nullable
in_alarm	bool	default false
last_triggered_at	timestamptz	nullable
last_notified_at	timestamptz	nullable
snoozed_until	timestamptz	nullable
version	int	optimistic concurrency

alert_event (append-only history)

column	type	notes
id	uuid (PK)	
alert_id	uuid (FK)	
triggered_at	timestamptz	
value	numeric	
message	text	

context	jsonb	corridor_id, amount_bucket, chart_id, etc.
notification_status	enum	`queued` / `sent` / `failed`
provider_safe	bool	true unless partner-permitted provider naming

Indexes

- `(alert_id, triggered_at desc)`
- `(triggered_at desc)` for recent activity dashboards

2.4 Notifications (prefs + unsubscribe + SES suppression)

notification_pref

column	type	notes
owner_type	enum	`user` / `guest`
user_id	uuid	nullable
guest_id	uuid	nullable
channel	enum	`email`
timezone	text	default `UTC`
daily_send_hour	int	default 9
digest_enabled	bool	default true
marketing_opt_in	bool	default false
unsubscribed	bool	default false
created_at	timestamptz	
updated_at	timestamptz	

Constraint

- Exactly one of `(user_id, guest_id)` must be non-null.

email_suppression (SES bounce/complaint safety)

column	type	notes
email_hash	text (PK)	privacy-safe hash of normalized email
reason	enum	`bounce` / `complaint` / `manual`
created_at	timestamptz	

2.5 Guest alerts subsystem (double opt-in; isolated identity)

guest_subscriber

column	type	notes
id	uuid (PK)	
email	citext (unique)	case-insensitive
status	enum	`pending` / `active` / `unsubscribed`
verify_token_hash	text	store hash only
unsubscribe_token_hash	text	store hash only
created_at	timestamptz	
verified_at	timestamptz	nullable
unsubscribed_at	timestamptz	nullable

source	text	e.g. `RateAlertForm`

Guest limitations (recommended)

- Frequency: **daily only**
- Strict quotas: e.g. 1 corridor + 1 rule
- No exports, no long history

2.6 History + Exports (async jobs)

export_job

column	type	notes
id	uuid (PK)	
user_id	uuid (FK)	
job_type	enum	`history_csv`, `history_pdf`, `pulse_csv`
params	jsonb	corridor_id, range, granularity, etc
status	enum	`queued`, `running`, `done`, `failed`
s3_key	text	nullable
expires_at	timestamptz	signed URL TTL
created_at	timestamptz	
started_at	timestamptz	nullable
finished_at	timestamptz	nullable
error	text	nullable

Indexes

- `(user_id, created_at desc)`
- `(status, created_at)` for worker pickup / admin

2.7 Telemetry (privacy-safe hot window)

telemetry_search_event

column	type	notes
id	uuid (PK)	
ts	timestamptz	
anon_session_id	text	nullable (random ID; do not store IP)
user_id	uuid	nullable
corridor_id	text	
amount_bucket	int	
payin	text	
payout	text	
utm	jsonb	nullable
page_path	text	nullable

telemetry_outbound_click

column	type	notes
id	uuid (PK)	
ts	timestamptz	
anon_session_id	text	nullable
user_id	uuid	nullable
provider_id	text	

corridor_id	text	
target_url	text	store sanitized/normalized
page_path	text	
utm	jsonb	nullable

Privacy invariants

- Store `anon_session_id`, not IP.
- Any published “intent index” must apply **Rule-of-5** (see Gold).

2.8 Contact + Newsletter (double opt-in)

contact_submission

column	type	notes
id	uuid (PK)	
email	citext	
name	text	nullable
message	text	
meta	jsonb	utm, page, context
created_at	timestamptz	

newsletter_subscriber

column	type	notes
id	uuid (PK)	
email	citext (unique)	
status	enum	`pending` / `active` / `unsubscribed`

verify_token_hash	text	
unsubscribe_token_hash	text	
created_at	timestamptz	
verified_at	timestamptz	nullable
unsubscribed_at	timestamptz	nullable
source	text	

---

3) Gold Layer — ClickHouse (Analytical)

3.1 PIT design: reference_time vs knowledge_time

- **reference_time:** when the market/provider quote is considered "true" (observation window)
- **knowledge_time:** when we learned it / computed it / corrected it

Restatement policy

- ✅ We do **not** update rows in place.
- Fixes become **new rows** with later `knowledge_time`.

As-of query pattern (conceptual)

- For each key, select the latest `knowledge_time` at or before the as_of timestamp.

3.2 Gold split: internal vs export

gold_internal

- Can be more granular (still never exposed externally).
- Used for debugging, internal analytics, model building.

gold_export

- **Only** datasets eligible for external use (B2B, exports, high-value APIs).
- Must satisfy:
  - Rights: Allowed_B2B
  - N≥3 contributors per published point
  - dominance checks

- timestamp bucketing + rounding/quantization
- suppression for small coverage / privacy

3.3 Core Gold fact shapes (conceptual)

corridor_index_fact (PIT-safe derived fact)

column	type	notes
corridor_id	String	
reference_time	DateTime	
knowledge_time	DateTime	
metric_name	String	e.g. `RCI`
metric_value	Float64	
contributor_count	UInt16	
dominance_metrics	String/JSON	optional
algorithm_version	String	
publisher_run_id	String	
publish_status	Enum	`published` / `quarantined` / `internal_only`

gold_export.cdp_daily (Corridor Daily Panel; export-safe)

Grain: (date, corridor_id, amount_bucket, payin, payout)

Recommended columns (representative):

- `rci_leader_bps`, `rci_median_bps`, `rci_p10_bps`, `rci_p90_bps`
- `dispersion_bps`, `leader_edge_bps`, `volatility_7d`
- `provider_count_binned` (bucketed, not exact small counts)
- `freshness_p50_sec`, `freshness_p95_sec`
- `suppression_flag`, `suppression_reason`

- `methodology_version`, `pipeline_version`

gold_export.cdp_4h

Same shape as `cdp_daily`, but with `time_bucket_utc` (4-hour buckets).

gold_export.mel (Market Events Log; derived)

- `time_bucket`, `corridor_id`, `event_type`, `severity`
- `deltas` (bucketed/abstracted), `notes` (non-source revealing)

gold_export.intent_index (Rule-of-5 gated)

Grain: (day, corridor_id)

- `normalized_index` (0–1)
- `volume_bucketed`
- `rule_of_5_applied` flag
- suppressed when < 5 distinct sessions/users

3.4 Polymorphic FX schema (multi rate regimes)

We model multiple rate regimes for the same currency pair:

- `official`, `blue`, `ccl`, `crypto_implied`, `mid_market`

fx_rate_fact (PIT-safe)

column	type	notes
base_currency	String	
quote_currency	String	
rate_type	Enum	official/blue/ccl/crypto_implied/mid_market
reference_time	DateTime	
knowledge_time	DateTime	
rate_value	Float64	
source	String	provider/api/exchange

quality_flags	String/JSON	
algorithm_version	String	if derived

FX invariants

- `rate_type` is explicit (no ambiguous “the FX rate”).
- PIT semantics apply.
- Downstream methodologies must state which `rate_type` they use.

---

4) Crosswalk — Frontend gaps → backend endpoints/jobs/tables

This is the direct mapping from frontend modules to backend deliverables.

A) Persist watchlists/alerts (today local)

- **Frontend:** `useWatchlist.ts`, `useAlerts.ts`, `tracking.ts` WatchTarget types
- **Backend:** Watchlist + Alerts Service (Aurora + APIs)

Build

- Tables: `watchlist_item`, `alert_rule`, `alert_state`, `alert_event`
- Endpoints:
  - `GET/POST/PATCH/DELETE /api/watchlist`
  - `GET/POST/PATCH/DELETE /api/alerts`

B) Alert evaluation + notifications + prefs

- **Frontend:** `dashboard.vue`, `SaveAlertModal.vue`
- **Backend:** Alert Evaluator + Notification Worker (EventBridge → SQS → Lambda/ECS + SES)

Build

- Scheduler (realtime/hourly/daily) enqueues evaluation tasks
- Worker reads: `latest_quote_by_provider` and/or `gold_export.*` and/or FX
- Writes: `alert_event`, updates `alert_state`
- Prefs: `notification_pref`, suppression: `email_suppression`

C) Guest alerts (email-only form)

- **Frontend:** `RateAlertForm.vue`
- **Backend:** Guest subsystem (double opt-in)

Build

- Table: `guest_subscriber` (+ watchlist/alerts via `guest_id`)
- Endpoints:
  - `POST /api/guest/alerts` (creates pending + sends verify email)
  - `GET /api/guest/confirm`
  - `GET /api/guest/unsubscribe`

D) Replace local entitlements/auth with real sessions + Stripe

- **Frontend:** `useAuth.ts`, `useEntitlements.ts`, `checkout.vue`, `success.vue`
- **Backend:** Identity/Billing/Entitlements Service

Build

- Tables: `user_plan`, `plan_usage_counters`, `billing_webhook_event`
- Endpoints:
  - `GET /api/me`
  - `POST /api/billing/checkout-session`
  - `POST /api/billing/webhook`
  - `POST /api/billing/portal` (optional)

E) Dashboard history/export tabs backed by real data

- **Frontend:** `dashboard.vue` tabs
- **Backend:** History + Exports Service (ClickHouse + S3 exports)

Build

- Endpoints:
  - `GET /api/history/corridor` (Gold)
  - `POST /api/exports` (create job)
  - `GET /api/exports/:id` (status)

- `GET /api/exports/:id/download` (signed URL)
- Tables: `export_job`

F) Replace mocks + add FX feed + persist telemetry + contact/newsletter

- **Frontend:** `providers.get.ts`, `bank-vs-specialist.get.ts`, `pulseMockApi.ts`, `recent-searches.post.ts`, `click.post.ts`, `contact.vue`, `NewsletterSignup.vue`

Build

- Endpoints:
  - `GET /api/quotes/current` (from `latest_quote_by_provider`)
  - `GET /api/pulse/overview`, `GET /api/pulse/chart/:id` (from `gold_export.*`)
  - `GET /api/fx/midmarket` (internal only)
  - `POST /api/telemetry/recent-search`
  - `POST /api/telemetry/click`
  - `POST /api/contact`
  - `POST /api/newsletter/subscribe` (double opt-in)
- Tables: telemetry + contact/newsletter tables above

---

5) Pipelines & jobs that must be rock-solid

5.1 Alert evaluation (realtime/hourly/daily)

Evaluation sources (metrics must come from ONE of):

- B2C current quotes: `latest_quote_by_provider` (or a "best quote view")
- Gold aggregates: `gold_export.cdp_4h` / `gold_export.cdp_daily`
- Mid-market rate: FX service

Scheduling

- Realtime (Plus): every 5 minutes (dedupe + cooldown makes this safe)
- Hourly: EventBridge hourly → enqueue corridors/users with hourly alerts
- Daily (Free + guests): EventBridge daily at user preferred hour (bucket by hour)

Worker writes

- Insert `alert_event`
- Update `alert_state` (versioned)
- Enqueue email send (do not block evaluation on SES)

Before sending

- Check `notification_pref.unsubscribed`
- Check `email_suppression`
- For guests: `guest_subscriber.status == active`

5.2 Exports pipeline (async; Gold only)

Flow

1. `POST /api/exports` creates `export_job(status=queued)`
2. Worker reads Gold export-safe tables
3. Writes file to S3 exports bucket
4. Updates `export_job` with `s3_key`, `expires_at`, status

Invariant

- Exportable datasets originate from **Gold export tables only**.

5.3 FX ingest (licensed mid-market)

- Ingest frequency: every 1–5 minutes (as needed)
- Store:
  - hot latest in Aurora (optional) for operational alignment
  - PIT time-series in Gold (`fx_rate_fact`)
- Track provenance: source, timestamp, freshness SLOs

---

6) Retention + lifecycle notes (operational guidance)

- `quote_record`: keep hot window in Aurora (e.g., 30–180 days); older history lives in Gold/Bronze.

- `export_job` rows: keep 90–180 days for audit; S3 files TTL via lifecycle policy (e.g., 7–30 days).
- `telemetry_*`: keep hot window (e.g., 30–90 days) in Aurora; optionally stream to Bronze for long-term.
- `guest_subscriber` + `newsletter_subscriber`: retain while active; remove/anonymize on request (privacy policy).
- `billing_webhook_event`: retain long-term (audit + debugging), with payload redaction as needed.

📊 Methodologies

This page defines the computational and product contracts for Remit-Scout's signals and user-facing metrics (what we compute, from what inputs, at what granularity, and under what safety gates).

Change Control (Required)
Any change to formulas, eligibility rules, thresholds, rounding, or what leaves Gold requires:

- ADR (what changed + why)
- Backfill/restatement plan (append-only, PIT-safe)
- Gold Publisher review (derived/irreversible boundary)
- Legal/compliance review if it impacts rights posture, reversibility, or provider attribution

---

4.0 Non-Negotiables (Hard Constraints)
These are invariants across every methodology and every downstream feature:

- No login scraping (no account creation, no authenticated sessions).
- No CAPTCHA solving, bypass techniques, or "cat-and-mouse" evasion loops.
- Stop-on-block: if blocked (captcha/403/429/anti-bot messaging), we stop collection and escalate.
- B2B outputs are derived-only, irreversible, and must pass Publisher gates:
  - N≥3 distinct contributing providers per published point
  - dominance checks
  - bucketing/rounding/suppression where needed
- If there is ambiguity, we default to withhold rather than publish.

4.1 RCI — Remittance Cost Index (All-in Cost)

Goal
Estimate the all-in cost of remitting on a corridor by combining:

- explicit fees
- FX margin vs a reference rate (mid-market or approved baseline)

Quote-level definitions

Given a quote:

- Send amount: S (send currency)
- Fee: F (normalized into send currency)
- Receive amount: R (receive currency)

Define:

- Total paid: P = S + F
- Effective rate: r_eff = R / P (receive per send)
- Reference FX: r_ref = FX_midmarket(send→receive, reference_time)

Then:

- FX margin (fraction): m_fx = 1 − (r_eff / r_ref)
- All-in cost (fraction): c_allin = m_fx

Optional reporting (if needed, but keep c_allin consistent):

- fee_share = F / P

Express in basis points:

- RCI_bps = c_allin × 10,000

Corridor aggregation (Corridor-bucket RCI)

Compute corridor-day (or corridor-hour / corridor-4h) RCI as an aggregate across providers:

- Eligibility:
  - contributing providers must be allowed for the relevant use (B2C display or B2B derived publishing)
  - stale/blocked sources excluded by policy
- Publisher gates (required for any external/published output):
  - N≥3 distinct independent providers per point
  - dominance checks (max share + top-2 share thresholds)
  - freshness bounds for the reference_time window
  - rounding/quantization to reduce reverse-engineering risk
- Robust aggregate:
  - default: median of provider-level RCI_bps (recommended)
  - alternative: trimmed mean (document trim %)

Outputs (Gold)

For each published point write:

- corridor_id, reference_time (bucketed), knowledge_time
- metric_name = RCI_bps, metric_value
- contributor_count
- dominance metrics (optional)
- algorithm_version + methodology_version
- publisher_run_id
- publish_status (published/quarantined/internal_only)

Notes / edge cases

- Fee currency normalization is mandatory (define normalization source + rounding rules).
- Promotions/fee waivers must be represented explicitly (F may be 0, but still audited).
- Any change to r_ref definition (mid-market provider, refresh rate, alignment window) triggers an ADR + restatement plan.

4.2 ISER — Implied Shadow Exchange Rate

Goal

Infer a shadow FX rate when official rates diverge from accessible market mechanisms using permitted public signals, such as:

- crypto P2P stablecoin pricing (e.g., USDT in local currency)
- MTO spreads / implied conversion paths
- other approved public signals

Method outline (conceptual)

1. Gather permitted reference prices:
   - local_currency_per_USD (from approved P2P/market sources)
2. Convert into implied FX for corridor currencies:
   - compute implied pairwise rates using consistent reference_time buckets
3. Apply quality filters:
   - minimum coverage / liquidity proxies
   - outlier rejection
   - PIT correctness (knowledge_time)
4. Publish only derived aggregates:

- never publish raw order book prints
- enforce Publisher gates (N≥3 where applicable, dominance/rounding/suppression)

Outputs (Gold)

- time_bucket_utc, currency/country, iser_rate_value
- confidence / quality_flags
- algorithm_version, methodology_version
- publish_status

4.3 TEER — Total Effective Exchange Rate (Composite)

Goal
Produce a composite exchange rate for a corridor reflecting a weighted blend of:

- observed remittance effective rates (from QuoteRecord / derived “best quote” views)
- regime FX rates (official/blue/ccl/crypto_implied/mid_market) as applicable

Weighting logic (template)
Define weights w_i that sum to 1, based on one or more:

- provider coverage / reliability
- corridor significance
- (optional) estimated volume proxies (only if explicitly allowed)

TEER(reference_time) = Σ w_i × rate_i(reference_time)

Rules

- Explicitly document which rate types are eligible inputs
- Enforce dominance constraints so TEER is not single-source
- Version methodology changes and restate via append-only rows (no updates in place)

Outputs (Gold)

- corridor_id, reference_time, knowledge_time
- teer_value (and optionally components metadata internally)
- contributor_count + dominance metrics
- methodology_version, algorithm_version
- publish_status

4.4 The Missing Layer: Product Backbone Services (App Contracts)

Purpose

The “Truth Engine + Lakehouse + Derived Boundary” is necessary but not sufficient. The Product Backbone turns the frontend into a real application by providing stable, first-class services for identity, entitlements, watchlists, alerts, exports, FX reference, telemetry, and marketing intake.

Backend modules (AWS)

1. Identity + Billing + Entitlements Service

- Supabase auth (JWT verification)
- Stripe checkout + webhooks
- Plan enforcement + quotas (Free/Plus/B2B)
- /api/me as single source of truth

2. Watchlist + Alerts + Notifications Service

- Persist watchlists & alerts
- Alert evaluation workers (realtime/hourly/daily)
- Alert history, notification prefs, unsubscribe handling
- Guest alert support (double opt-in)

3. History + Exports Service

- Time-series backed by Gold (ClickHouse)
- Export job queue (CSV/PDF) to S3 + signed downloads

4. FX Mid-Market Rate Service

- Ingest licensed mid-market feed
- Store aligned rates used for QuoteRecord computation + charting + RCI correctness

5. Telemetry Ingestion Service

- Persist recent searches + outbound clicks
- Feed “Intent” indices (with privacy gating and suppression rules)

6. Contact + Newsletter Ingestion

- Store submissions
- Double opt-in for marketing/newsletter lists
- Optional ESP push later (Mailchimp/etc.)

- Sponsored placements are **presentation metadata** and **do not change** computation, ranking, or any derived metrics.
- Publisher gates apply to **datasets leaving Plane C**, not to sponsored UI blocks—however sponsored providers must still comply with Allowed_B2C and never reveal raw data.

4.5 Crosswalk: Frontend Gaps → Backend Endpoints / Jobs / Tables

A) Persist watchlists/alerts (today local)
Frontend: useWatchlist.ts, useAlerts.ts
Backend: Watchlist + Alerts Service (Aurora Postgres + APIs)
Build:

- watchlist_item (target_type + target_payload_jsonb to cover all WatchTarget variants)
- /api/watchlist CRUD
- /api/alerts CRUD (linked to watchlist items)

B) Alert evaluation + notification pipeline + prefs
Frontend: dashboard.vue, SaveAlertModal.vue
Backend: Evaluator + Notification workers (EventBridge → SQS → Lambda/ECS + SES)
Build:

- scheduler for realtime/hourly/daily
- alert_event history
- notification_pref + unsubscribe support
- SES bounce/complaint suppression list

C) Guest alerts (email-only)
Frontend: RateAlertForm.vue
Backend: guest subsystem (double opt-in)
Build:

- guest_subscriber (pending→active→unsubscribed)
- /api/guest/alerts create (returns “check email to confirm”)
- /api/guest/confirm + /api/guest/unsubscribe

D) Replace local entitlements/auth with real sessions + Stripe
Frontend: useAuth.ts, useEntitlements.ts, checkout.vue, success.vue
Backend: Identity/Billing/Entitlements Service
Build:

- /api/me returns plan + entitlements + quotas
- /api/billing/checkout-session
- /api/billing/webhook
- /api/billing/portal (optional)

E) Dashboard history/export tabs backed by Gold + export jobs
Frontend: dashboard.vue tabs
Backend: History + Exports Service
Build:

- /api/history/corridor (Gold)
- /api/exports create job + status + download
- plan gating + quotas enforced server-side

F) Replace mocks + add FX + telemetry + contact/newsletter
Frontend: providers.get.ts, bank-vs-specialist.get.ts, pulseMockApi.ts, recent-searches.post.ts, click.post.ts, contact.vue, NewsletterSignup.vue
Backend: Quote API + Pulse API + FX Service + Telemetry + Marketing Intake
Build:

- /api/quotes/current (Aurora latest_quote_by_provider)
- /api/pulse/overview + /api/pulse/chart/:id (ClickHouse Gold)
- /api/fx/midmarket (internal) + ingestion job
- /api/telemetry/recent-search + /api/telemetry/click
- /api/contact + /api/newsletter/subscribe (double opt-in)

4.6 Concrete Operational Data Model (Aurora Postgres)

Watchlists (future-proof via JSONB)
watchlist_item

- id (uuid pk)
- owner_type (user|guest)
- user_id (uuid, nullable)
- guest_id (uuid, nullable)
- target_type (text) // corridor, provider, pulse_metric, custom, ...
- target_payload (jsonb) // corridor_id, amount_bucket, payin/payout, provider_id, chart_id, ...
- label (text, nullable)

- created_at, updated_at, deleted_at

  Constraint: exactly one of (user_id, guest_id) must be non-null.

Alerts

alert_rule

- id, watchlist_item_id
- metric (enum)
- comparator (enum)
- threshold (numeric)
- frequency (realtime|hourly|daily)
- cooldown_minutes (default 360)
- enabled
- created_at, updated_at

alert_state

- alert_id (pk/fk)
- last_evaluated_at, last_value, in_alarm
- last_triggered_at, last_notified_at
- snoozed_until
- version (for safe concurrent updates)

alert_event (history)

- id, alert_id, triggered_at, value
- message, context (jsonb)
- notification_status (queued|sent|failed)
- provider_safe (bool; default true)

Notification prefs + suppression

notification_pref

- owner_type, user_id/guest_id
- channel (email)
- timezone, daily_send_hour
- digest_enabled, marketing_opt_in
- unsubscribed
- created_at, updated_at

email_suppression

- email_hash (pk)
- reason (bounce|complaint|manual)
- created_at

Guest alerts (double opt-in)

guest_subscriber

- id, email (unique)
- status (pending|active|unsubscribed)
- verify_token_hash, unsubscribe_token_hash
- created_at, verified_at, unsubscribed_at
- source

Entitlements + Stripe cache

user_plan

- user_id (pk)
- plan_code (free|plus|enterprise)
- status (active|past_due|canceled|trialing)
- stripe_customer_id, stripe_subscription_id
- current_period_end, cancel_at_period_end
- created_at, updated_at

plan_usage_counters (recommended)

- user_id (pk)
- period_start, period_end
- alerts_count, exports_count, api_calls_count
- updated_at

Exports (async)

export_job

- id, user_id
- job_type (history_csv|history_pdf|pulse_csv)
- params (jsonb)
- status (queued|running|done|failed)
- s3_key, expires_at

- created_at, started_at, finished_at
- error

Telemetry (privacy-safe)
telemetry_search_event

- id, ts
- anon_session_id (nullable), user_id (nullable)
- corridor_id, amount_bucket, payin, payout
- utm (jsonb), page_path

telemetry_outbound_click

- id, ts
- anon_session_id (nullable), user_id (nullable)
- provider_id, corridor_id
- target_url (sanitized/normalized), page_path
- utm (jsonb)

Contact + newsletter
contact_submission

- id, email, name, message
- meta (jsonb), created_at

newsletter_subscriber

- id, email (unique)
- status (pending|active|unsubscribed)
- verify_token_hash, unsubscribe_token_hash
- created_at, verified_at, unsubscribed_at
- source

4.7 Alert Evaluation & Notification Pipeline (Realtime/Hourly/Daily)

Evaluation sources (what value are we checking?)
Alert metrics must be computed from one of:

- B2C current quote views (Aurora latest_quote_by_provider / “best quote” view)
- Gold aggregates (ClickHouse gold_export.* series)
- Mid-market FX service (for mid_market_rate and RCI correctness)

Scheduling design

- Realtime (Plus):
  - practical default: every 5 minutes for corridors with active realtime alerts
- Hourly:
  - EventBridge hourly → enqueue tasks per corridor
- Daily (Free + guests):
  - EventBridge daily at user preferred hour (bucketed) → enqueue tasks

Worker logic (anti-spam and correctness)
For each alert:

1. Fetch current metric value
2. Evaluate comparator
3. Apply cooldown and “crosses_above/below” using last_value
4. Write alert_event + update alert_state
5. Queue email send via SES (do not block evaluation)

Before sending notifications

- check notification_pref.unsubscribed
- check email_suppression
- check guest status == active
- enforce plan quotas (server-side)

4.8 Guest Alerts: Decision & Constraints

Decision
Support guest alerts, but isolate them cleanly.

Why

- frontend already has email-only flows
- strong acquisition loop
- can be implemented safely with double opt-in

Constraints (recommended)

- daily frequency only
- strict quotas (e.g., 1 corridor + 1 rule)
- no exports

- no long history

4.9 Auth + Entitlements + Stripe

/api/me is the contract
Return:

- user_id, email
- plan_code, status
- entitlements + quotas:
  - history_max_days
  - alerts_max
  - watchlist_max
  - exports_enabled
  - pulse_access_level
    Rule: server enforces gating on every request; frontend gating is UX only.

Stripe integration (minimal but complete)

- POST /api/billing/checkout-session
- POST /api/billing/webhook
- POST /api/billing/portal (optional)
  Webhook writes into user_plan and updates entitlements cache.

4.10 History + Exports Backed by Gold4.11 Provider Quality Score — RemitScore

Purpose
RemitScore is a user-facing provider quality rating displayed as a 1–10 score (e.g., 9.3/10). It is not a price metric; it is a supplementary quality signal intended to help users interpret tradeoffs.

Inputs
RemitScore is composed from a small set of category scores we maintain per provider:

- Delivered Value (Pricing) – relative cost competitiveness (fees + FX margin).
- Speed & Friction – transfer speed and UX friction combined.
- Reliability – operational consistency and availability.
- Support & Refunds – support quality and dispute/refund posture.
- Trust & Safety – safety/regulatory/reputation considerations.

Computation (Versioned)

RemitScore MUST be versioned (methodology_version) and treated as a governed formula. Any changes require ADR + restatement plan per Change Control.

- v1: weighted aggregation of the category scores, normalized to a 10-point scale.
- The exact weights MUST be explicitly documented here once confirmed from the canonical implementation (frontend or backend), and must not be “implied”.

Operational notes

- RemitScore may be static / research-driven initially (manually maintained values). If/when it becomes dynamic, the data source and update cadence must be documented.
- RemitScore must not override default ordering of quote results unless the user explicitly sorts by rating.
- RemitScore is safe to publish because it is not raw provider quote data; however it is still a user-facing contract and must be stable and versioned.

4.12 Quote Comparison Contract — Ranking, Best Flags, and Mid-Market Fields

This section defines the product contract for how quotes are compared and what derived fields are returned to the frontend.

Primary ranking (default)

- Default sort is most received (best value to recipient), which is equivalent to lowest total cost under the same send amount and corridor constraints.
- If two quotes are effectively tied, apply deterministic secondary ordering (e.g., faster delivery time, then provider name) to keep UI stable.

Derived fields per quote

For each provider quote, the API returns:

- All-in cost signal (e.g., RCI in bps) consistent with section 4.1.
- FX markup fields derived using the mid-market reference rate used for that request:
  - fxMarkupBps (basis points vs mid).
  - fxMarkupPercent (percentage vs mid).
  - deltaVsMidMarket (optional UI-friendly delta in receive currency).

Best flags (computed per response)

The API marks “best-of” badges used by the UI:

- cheapest: provider with lowest total cost (best received amount).

- fastest: provider with shortest delivery time (where available).
- topRated: provider with highest RemitScore in the returned set.
- bestValue: optional; if used, must be explicitly defined (e.g., Pareto frontier or a cost/speed composite rule).

These flags are computed fresh per response and must be deterministic.

Corridor insights / recommendations
If included in the response (for the UI's "insights" panels), these are derived from Gold aggregates and must:

- respect publisher gates (N≥3, dominance checks, rounding/suppression).
- be parameterized by a documented lookback window (e.g., last 30 days).
- be clearly labeled as historical/aggregate context, not "live" quote facts.

4.13 Pulse "Opportunity" Definition

Pulse highlights "opportunities" when the current best option is unusually favorable versus recent history.

Definition (v1)
A corridor is flagged as an "opportunity" if:

- the current best all-in cost (RCI) is meaningfully lower than a rolling historical baseline (e.g., 7-day rolling average), and
- the delta exceeds a documented threshold (tunable), and
- the output passes Publisher gates (N≥3, dominance checks, rounding/suppression).

Source of truth
The full Pulse contract (endpoints, chart types, caching, and refresh cadence) lives on the 📈 Pulse Analytics page.

History APIs

- GET /api/history/corridor reads ClickHouse gold_export.* (bucketed)
- plan-gated range (e.g., 30d free vs 365d plus)
- always derived + suppressed for small-N

Exports (async jobs)
Flow:

- POST /api/exports creates export_job + SQS message
- worker generates CSV/PDF from Gold series

- store in S3 exports bucket
- GET /api/exports/:id polls status
- GET /api/exports/:id/download returns a pre-signed URL

🛠️ Operations & Runbooks

Purpose

This page is the “Day-2 manual” for Remit-Scout. It exists to:

- minimize MTTR (fast incident triage + safe mitigations)
- keep us compliant under stress (stop > evade; withhold > publish)
- reduce technical debt by standardizing how we document + convert specs into plans and tasks

Scope

Covers the operational surface of:

- Plane A: Product Backbone + delivery APIs
- Plane B: Truth Engine ingestion + governance controls
- Plane C: Gold Publisher + derived datasets + exports
- Shared infra: AWS edge/security, databases, queues/streams, email, billing

Related docs (source of truth)

- 🏗️ System Architecture (mental model + plane boundaries)
- 🏛️ Compliance & Governance (rights matrix, gold publisher gates, proxy policy, legal escalations)
- 💾 Data Models & Schemas (tables, invariants, PIT semantics)
- 📊 Methodologies (RCI/ISER/TEER definitions + change control)

Operating Principles (Non-Negotiables)

1. Stop > evade

- If blocked (captcha/403/429/anti-bot), stop collection and escalate.
- No login scraping, no CAPTCHA solving, no bypass techniques, no “cat-and-mouse”.

2. Withhold > publish

- If ambiguous rights or reversibility risk: default to withhold.
- Raw data never leaves Plane B. Only Derived (Gold) leaves via Plane C.

3. Least privilege by plane

- External-facing services must not have credentials/network access to raw artifacts.
- B2B roles read gold_export only.

4. Auditability

- Every published metric must be traceable to Silver/Bronze lineage and have a publisher audit record.
- All ops actions must be logged (admin_audit_log / equivalent).

---

0. Quick Links & Owner Map

Ownership (default)

- On-call Engineering: first response + mitigation
- Data/Platform Owner: pipeline health + backfills/restatements
- Security Owner: abuse, WAF, bans, incident containment
- Compliance/Legal Owner: rights posture changes, C&D, takedown workflows
- Product Owner: customer comms + feature gating decisions

Critical consoles (fill with your URLs)

- CloudWatch Dashboards: [link]
- Logs (CloudWatch / OpenSearch): [link]
- Tracing (X-Ray / OTEL): [link]
- WAF dashboard + IP sets: [link]
- Aurora console + RDS Performance Insights: [link]
- ClickHouse console/monitoring: [link]
- SQS queues + DLQs: [link]
- Kinesis streams + consumer lag: [link]
- SES dashboards (bounces/complaints): [link]
- Stripe dashboard (webhooks + events): [link]
- Supabase (auth/JWT keys status): [link]
- Admin Console (stoplist/budgets/circuit breakers): [link]

---

1. Incident Management

1.1 Severity levels (use these consistently)

SEV0 — Complete outage / material compliance risk
Examples:

- Raw data exposure risk or suspected leak
- Gold Publisher gate bypassed or wrong publish occurred externally
- Major API outage for B2C (widespread 5xx), or billing/auth outage affecting all paid users

SEV1 — Major degradation / partial outage
Examples:

- Top corridors stale beyond SLO
- Multiple Tier-1 providers blocked simultaneously
- Alert system spamming users or failing to send to many users

SEV2 — Localized degradation
Examples:

- Single provider blocked
- Single service elevated latency
- Export jobs failing intermittently

SEV3 — Minor issue / backlog item
Examples:

- Cosmetic dashboard gaps
- Non-critical metrics missing

1.2 Incident roles

- Incident Commander (IC): owns timeline, assigns, decides mitigations
- Ops Lead: executes changes (stoplists, rollbacks, config)
- Comms Lead: internal updates + customer-facing message (if needed)
- Scribe: captures timeline + decisions for postmortem

1.3 First 10 minutes checklist (any SEV)

- Confirm severity (SEV0/1/2/3)
- Identify blast radius: B2C, B2B, ingestion, publish, billing, notifications
- Freeze risky changes:
  - disable deployments unless needed to mitigate
  - disable external publishing if integrity is in doubt (Gold Publisher “hold”)
- Capture evidence:

- logs (errors + correlation IDs), queue depth, last successful run IDs
- affected providers/corridors/time window

- Apply safe mitigations:
  - stoplist providers on block, throttle budgets, enable read-only mode for some endpoints, etc.

1.4 Postmortem requirements

- SEV0/SEV1 require postmortem within 48h:
  - root cause(s)
  - contributing factors
  - what detection missed
  - action items with owners + deadlines
  - prevention: tests, alarms, runbook updates

Postmortem template (copy/paste)

- Summary:
- Customer impact:
- Timeline:
- Root cause:
- Detection:
- Mitigation:
- Corrective actions:
- Preventative actions:
- Lessons learned:

---

2. Monitoring, SLOs, and Alerts

2.1 Health checks (service-level)

Every service should expose:

- /healthz = process up
- /readyz = dependencies reachable (Aurora/Redis/ClickHouse/queues)
- synthetic canaries hit critical APIs and validate schema + non-empty responses

2.2 Data health (more important than “service is up”)

Track at minimum:

- Freshness p50/p95 for top corridors (by amount_bucket + payin/payout)
- Quote success rate (15m, 1h) per provider + global
- Provider coverage (distinct providers contributing) per corridor bucket
- Pipeline lag:
  - SQS queue depth
  - Kinesis consumer lag / iterator age
  - Gold builder lag to last Silver ingest
- Suppression rates (Gold Publisher): % of points withheld + reasons
- Email metrics:
  - send success rate
  - bounce/complaint rate
  - unsubscribe spikes

2.3 Suggested SLO targets (adjust)

- Top corridors: p95 freshness ≤ 5 min (B2C “current”)
- Gold export lag: ≤ 15 min for intraday buckets, ≤ 2h for daily
- Quote success rate: ≥ 98% (rolling 1h) for Tier-1 sources
- Alert notifications: ≥ 99% delivered (excluding suppression/bounces)

2.4 Alerting rules (minimum)

- Freshness breach (top corridors)
- Provider block detection spike (captcha/403/429 patterns)
- Gold Publisher “publish failure” spike (N<3, dominance, staleness)
- Aurora: CPU high, connections high, replication lag
- ClickHouse: disk high, insert errors, query latency high
- SQS: queue depth high, DLQ growth
- Stripe webhook failures / backlog
- SES bounce/complaint spikes
- WAF rate-based blocks spike (possible attack)

---

3. Operational Control Plane

3.1 Stoplist & circuit breakers (governance-backed)

Rules:

- Stoplist_Status is enforceable by automation (scheduler must honor it).
- Any block signal → stop collection attempts and pause provider (per compliance policy).

Operational actions:

- Pause provider: set Stoplist_Status=Paused (or Legal Hold)
- Resume provider: only after verifying:
  - rights posture unchanged (Allowed_Collect / Allowed_B2B as relevant)
  - no block signals under conservative budget
  - parser stable (no QC failures)

3.2 Budgets & politeness controls
Maintain per-domain budgets:

- rpm_limit / rph_limit
- concurrency_cap
- crawl_delay_ms
- proxy country policy (localization-only)

Emergency throttle:

- Reduce schedule frequency for affected domains
- Decrease concurrency cap globally during incidents to protect infra and reduce provider load

3.3 Admin audit logging
Every manual change must write:

- who
- what changed (before/after)
- why (incident id / link)
- when

---

4. Runbook Index

Runbooks are written in a standard format:

- Trigger / Symptoms
- Fast triage (5 minutes)
- Mitigations (safe-first)
- Verification (exit criteria)
- Follow-up (prevent recurrence)

- References (dashboards/tables/configs)

RBK-COLLECT-001 Provider blocked / anti-bot signals (captcha/403/429)
RBK-COLLECT-002 Provider ToS change / rights uncertainty (same-day matrix update)
RBK-COLLECT-003 Parser/layout change causing bad quotes
RBK-PIPE-001 SQS backlog / DLQ growth (ingest or alerts)
RBK-PIPE-002 Kinesis consumer lag / stuck checkpoints
RBK-DB-001 Aurora high CPU / connection storm / failover
RBK-CH-001 ClickHouse high latency / insert failures / disk pressure
RBK-CACHE-001 Redis degraded / cache stampede / rate limiting failure
RBK-PUBLISH-001 Gold Publisher gate failures / suppression spike
RBK-PUBLISH-002 Restatement/backfill (PIT-safe append-only)
RBK-AUTH-001 Supabase JWT verification failures / JWK rotation
RBK-BILL-001 Stripe checkout or webhook failures / plan mismatch
RBK-EMAIL-001 SES bounces/complaints spike / suppression list correctness
RBK-ALERTS-001 Alert storm (spam prevention + disable path)
RBK-EXPORT-001 Export jobs stuck / S3 signed download failures
RBK-FX-001 Mid-market FX feed stale or drifting
RBK-SEC-001 API abuse / scraping attack / WAF + blocklist response
RBK-PRIV-001 Telemetry privacy gating / Rule-of-5 enforcement regression
RBK-REL-001 Deployment rollback / migration issue

---

5. Runbooks

RBK-COLLECT-001 — Provider blocked / anti-bot signals

Trigger / Symptoms

- Collector detects captcha page, 403/429 patterns, explicit anti-bot messaging
- Success rate drops sharply for a provider/domain
- Circuit breaker opens repeatedly

Fast triage (5 minutes)

- Confirm block signals in logs (response status codes + content fingerprints)
- Check if failures started after a deploy or after increasing frequency
- Identify affected providers/corridors and the last successful run_id

Mitigations (safe-first)

1. Immediately stop:
    - open circuit breaker
    - set Stoplist_Status=Paused for that provider/source method
2. Reduce risk:
    - reduce domain budget (rpm/concurrency) for adjacent providers on same domain (if shared)
3. Preserve evidence:
    - timestamps, run_ids, response metadata, proxy country used
4. Notify:
    - engineering on-call
    - if repeated or if provider is sensitive: compliance/legal owner

Verification / exit criteria

- Provider remains paused (no further scheduled attempts)
- Incident ticket created with evidence and proposed remediation plan
- If resuming: “probe” run under very conservative budget passes without blocks

Follow-up

- Update provider playbook: budgets, collector fingerprints, QC checks
- Consider partnership path if recurring blocks

References

- 🏛️ Compliance & Governance → Stop-on-Block Policy

RBK-COLLECT-002 — ToS change / rights uncertainty

Trigger / Symptoms

- New ToS detected that affects Allowed_Collect/Allowed_B2C/Allowed_B2B
- Provider emails a notice or changes robots/blocks
- Any ambiguity from legal guidance

Mitigations

- Default to WITHHOLD:
    - pause provider
    - prevent publishing outputs referencing provider (if needed)
- Update Data Rights Matrix same day

- Log links to ToS excerpt / correspondence and add Owner + Last_Reviewed_At

Verification

- Matrix updated and reviewed
- Scheduling honors stoplist
- Publishing gates reflect updated allowed flags

References

- 🏛️ Compliance & Governance → Data Rights Matrix + Legal Escalations

---

RBK-COLLECT-003 — Parser/layout change causing bad quotes

Trigger

- Quality_flags spike: suspected_layout_change, partial, missing fields
- Outliers/extreme deltas increase
- Provider “recipient gets” becomes zero/negative, fee missing, etc.

Fast triage

- Identify first failing parser_version + deployment correlation
- Compare Bronze/Silver for affected runs
- Confirm whether ingestion is blocked vs parsing broken

Mitigations

- Pause provider if correctness can’t be guaranteed
- If partial safe mode exists: collect but quarantine outputs (do not publish)
- Roll back parser if recent change introduced regression

Verification

- QC flags return to baseline
- Sample quotes validated manually across corridors

Follow-up

- Add regression tests using saved fixtures (Bronze structured captures)
- Improve anomaly guardrails

RBK-PIPE-001 — SQS backlog / DLQ growth

Trigger

- Queue depth grows continuously
- DLQ receives messages
- Downstream lag (Silver not updating, alerts not evaluating)

Fast triage

- Identify which queue: scrape tasks, normalizer, alerts, exports
- Check consumer health: Lambda errors/timeouts, Fargate task failures
- Look for “poison messages” causing repeated failures

Mitigations

- Scale consumers (increase concurrency) within cost bounds
- If poison messages: route to DLQ, alert owner, patch validator to quarantine
- If dependencies failing (DB down): pause producers to stop backlog growth

Verification

- Queue depth decreasing
- DLQ stabilized and investigated
- Data freshness recovers

RBK-DB-001 — Aurora high CPU / connection storm / failover

Trigger

- Elevated p95 latency for API endpoints reading Aurora
- High CPU, high connections, slow queries, replica lag
- RDS failover event

Fast triage

- Check RDS Performance Insights (top queries)
- Identify connection storms (Lambda concurrency spikes, missing RDS Proxy)
- Confirm if issue is read vs write pressure

Mitigations

- Enable/verify RDS Proxy (if using Lambda)
- Temporarily lower Lambda concurrency / shed non-critical traffic
- Increase caching TTL for public endpoints (short-term)

- Scale Aurora (bump instance size / add read replica) if needed

Verification

- Latency back to normal
- No ongoing failovers
- Connection count stabilized

Follow-up

- Add query indexes/materialized views (e.g., latest_quote_by_provider)
- Add circuit breakers for expensive endpoints

RBK-PUBLISH-001 — Gold Publisher gate failures / suppression spike

Trigger

- Sudden spike in “withheld/quarantined” points
- N<3 failures, dominance failures, staleness failures
- External dashboards missing data buckets

Fast triage

- Identify which gate is failing (N≥3 vs dominance vs freshness)
- Check upstream coverage (ingestion success rate) vs downstream compute bugs
- Check if a provider got paused (stoplist change) reducing N

Mitigations

- If coverage outage: do not relax gates; allow suppression and recover coverage
- If dominance failure: broaden aggregation window (where permitted) or withhold
- If compute bug: quarantine outputs and roll back aggregator version

Verification

- Gate pass rate returns to baseline OR suppression reasons explain outages
- No “unsafe publish” performed to mask issue

Follow-up

- Add alerts for gate pass rate and top suppression reasons
- Improve fallback windows per metric

RBK-AUTH-001 — Supabase JWT verification failures / JWK rotation

Trigger

- Spike in 401s for authenticated endpoints
- JWT verification errors (kid not found, signature invalid)

Fast triage

- Confirm Supabase status / auth service health
- Check if our JWK cache is stale
- Validate token structure from a sample request

Mitigations

- Refresh JWK keys (cache bust)
- If Supabase degraded: allow short, bounded grace period for already-valid sessions if policy permits (document explicitly)
- Temporarily degrade Plus features only (keep public endpoints up)

Verification

- 401 rate returns to baseline
- /api/me works for known test user

Follow-up

- Ensure JWK refresh strategy is robust (cache TTL + auto refresh on kid miss)

RBK-BILL-001 — Stripe checkout/webhook failures / plan mismatch

Trigger

- Users can’t upgrade, checkout fails
- Webhook delivery failures
- /api/me shows wrong plan vs Stripe

Fast triage

- Check Stripe webhook endpoint health + signature verification errors
- Check webhook backlog / retries in Stripe dashboard
- Inspect user_plan rows and last updated timestamps

Mitigations

- Fix webhook handler (signature, idempotency keys)
- Reconcile affected users:
  - pull Stripe subscription state
  - update user_plan + entitlements cache
- If widespread: temporarily disable upgrades and show status banner

Verification

- Webhook deliveries succeed
- /api/me returns correct plan_code + entitlements

Follow-up

- Add “billing reconciliation” job (daily) to catch drift

RBK-EMAIL-001 — SES bounce/complaint spike / suppression correctness

Trigger

- Bounce rate spikes, complaint rate spikes
- SES sending paused or reputation risk

Fast triage

- Identify campaign/source: alerts vs marketing vs double opt-in
- Inspect email_suppression table updates and SES feedback handling

Mitigations

- Immediately stop sends to affected segment (feature flag)
- Ensure bounce/complaint feedback writes to email_suppression
- Verify unsubscribe links and pref handling
- If required, throttle sends to reduce reputation damage

Verification

- Bounce/complaint back to normal
- Suppression rules correctly prevent re-sends

Follow-up

- Add alarms on bounce/complaint thresholds
- Add canary emails for deliverability checks

RBK-ALERTS-001 — Alert storm (spam prevention)

Trigger

- Sudden spike in alert_event creation or email sends
- User complaints of repeated alerts
- SES rate limits hit

Fast triage

- Identify if caused by data anomaly (bad values) vs logic bug (cooldown/crossing)
- Check cooldown enforcement and last_notified_at updates
- Identify affected metric(s) and corridor(s)

Mitigations

- Kill switch:
  - disable evaluation for the affected metric/frequency tier
  - OR enforce global “max sends per minute” guard
- Fix data anomaly:
  - quarantine bad buckets
  - roll back methodology version if necessary
- Ensure email_suppression and unsub checks run before send

Verification

- Send rate stabilizes
- New alert_event rate returns to baseline

Follow-up

- Add stronger hysteresis/cooldown tests and replay simulations

---

RBK-EXPORT-001 — Export jobs stuck / S3 signed downloads failing

Trigger

- export_job stuck in queued/running
- download endpoint errors or signed URL invalid

Fast triage

- Check queue depth for exports
- Check worker logs (ClickHouse query timeout, S3 PutObject errors)

- Validate S3 key and lifecycle policies

Mitigations

- Retry jobs with exponential backoff
- If ClickHouse pressure: throttle exports concurrency
- If S3 signing issue: verify IAM perms + URL generation + clock skew

Verification

- Jobs complete end-to-end
- Signed URL works within TTL

Follow-up

- Add per-plan export rate limits and worker concurrency caps

---

RBK-FX-001 — Mid-market FX feed stale or drifting

Trigger

- FX feed last_updated > threshold
- RCI computations deviate unexpectedly across corridors
- Downstream metrics show discontinuities not explained by market

Fast triage

- Check FX ingest job health + provider status
- Validate a few reference pairs manually (spot-check)
- Confirm alignment logic (bucket/reference_time)

Mitigations

- Fail closed for derived metrics requiring FX reference (withhold publish if correctness risk)
- Switch to backup FX source (if approved)
- Re-ingest missing interval and restate affected Gold rows

Verification

- FX tables up to date
- Methodology outputs stable and within expected bounds

Follow-up

- Add FX freshness alarms; add dual-source validation if permitted

RBK-SEC-001 — API abuse / scraping attack against Remit-Scout

Trigger

- WAF rate-based blocks spike
- Redis rate limiter hot keys spike
- API p95 latency increases with unusual traffic patterns

Fast triage

- Identify top IPs/ASNs/paths
- Confirm whether traffic hits high-value endpoints (pulse/history/embed)

Mitigations

- Apply WAF rules:
  - tighten rate-based blocks
  - enable challenge for ambiguous requests
  - add IP/CIDR to WAF IP set (temporary with expiry)
- App-level:
  - lower limits per endpoint class
  - enforce signed tokens for embeds
  - require auth for certain endpoints if policy allows

Verification

- Latency recovers
- Abuse traffic reduced
- Legit traffic unaffected (watch false positives)

Follow-up

- Add “blocklist” workflow + admin console UI
- Document evidence and reasons (DDQ-friendly)

---

6. Release Management Runbook (RBK-REL-001)

Pre-deploy checklist

- No active SEV incidents
- Dashboards healthy (freshness, errors, queue lag)

- Migrations reviewed and rehearsed in staging
- Rollback plan written

Deploy steps

- Deploy to staging
- Run smoke tests:
  - /api/quotes/current
  - /api/me (auth)
  - /api/pulse/overview
  - small export job
  - alert evaluation dry-run (no sends)
- Deploy to prod with canary traffic (if supported)
- Monitor error rate and latency for 30 minutes

Rollback

- Revert code version
- If schema migration broke things:
  - apply reverse migration if safe OR
  - restore from snapshot in staging and patch forward; avoid restoring prod unless last resort

Post-deploy validation

- Data freshness OK
- Gold pipeline lag OK
- Stripe + SES integrations OK

---

7. Backups & Disaster Recovery

Aurora (Silver)

- Multi-AZ enabled
- PITR enabled
- Snapshot retention policy documented
- Monthly restore drill into staging:
  - restore snapshot
  - run read/write smoke tests
  - validate key tables and endpoints

ClickHouse (Gold)

- Backups to S3
- Restore drill quarterly
- Daily “gold_export” snapshot export (optional but recommended)

S3

- Versioning enabled
- Lifecycle policies documented (Bronze, exports)
- Access logs enabled
- Critical exports: optional cross-region replication (if RTO requires)

---

8. “Product Backbone” Ops Surface (What we operate day-to-day)

Identity + Billing + Entitlements

- Primary contract: /api/me
- Ops focus:
  - Stripe webhook reliability
  - entitlement drift detection + reconciliation
  - plan quota enforcement

Watchlists + Alerts + Notifications

- Ops focus:
  - alert evaluation health (queue depth, send rate)
  - spam prevention controls (cooldowns, global caps, kill switch)
  - suppression + unsub correctness (SES feedback loops)

History + Exports

- Ops focus:
  - ClickHouse query performance
  - exports worker capacity
  - S3 signed downloads reliability

FX Mid-Market Rate Service

- Ops focus:
  - feed freshness

- drift detection
- restatement workflow for FX-aligned metrics (RCI)

Telemetry Ingestion

- Ops focus:
  - ingestion not impacting core latency
  - privacy gating (Rule-of-5)
  - abuse protection (rate limit + sampling)

Contact + Newsletter

- Ops focus:
  - double opt-in deliverability
  - suppression + unsub compliance
  - spam filtering and rate limits

---

9. Turning Confluence Documentation into Plans & Tasks (LLM-friendly workflow)

Goal
Make docs “executable”: each section should map cleanly to epics, tickets, and acceptance tests.

9.1 Canonical page structure (use everywhere)
For any backend module or pipeline, include these headings in this order:

- Purpose
- Scope / Non-goals
- Interfaces (APIs, events, jobs)
- Data model (tables + invariants)
- Dependencies (Aurora/ClickHouse/SES/Stripe/etc.)
- SLOs and dashboards
- Failure modes + runbooks
- Change control (ADR required? backfill needed?)
- Open questions

9.2 Make every item “taskable” with explicit acceptance criteria
When you describe a deliverable, always include:

- What endpoint/table/job exists

- What “done” means (acceptance criteria)
- What is monitored (metric + alarm)
- What is the rollback / safe failure mode

Example “task block” (copy/paste)

- Deliverable:
- Owner:
- Dependencies:
- Acceptance criteria:
- Observability:
- Rollback plan:
- Risk/compliance notes:
- Links (schema/migration/ADR/runbook):

9.3 Convert this doc into a backlog (repeatable approach)

Step A — Identify bounded contexts (epics)

- Billing/Entitlements
- Watchlists/Alerts
- History/Exports
- FX Service
- Telemetry
- Marketing Intake
- Ops Console

Step B — For each epic, create “thin vertical slices”

Each slice must include:

- API contract + auth
- minimal table(s)
- minimal worker/job
- metric/alarm + runbook entry
- integration test (smoke)

Step C — Require ADRs for “boundary” changes

If it changes:

- rights posture

- publisher gates / irreversibility
- PIT semantics / restatement behavior
  → ADR required + backfill plan.

9.4 Where LLMs help (and how we keep output safe)
Use LLMs to:

- draft runbooks from known failure modes
- generate ticket templates from structured spec blocks
- produce checklists (deploy, go-live, DR drill)

Do NOT use LLM output as-is for:

- legal interpretation
- bypass/circumvention strategies
- “just ship” changes to publisher gates or rights posture

---

10. Production Readiness Checklist (Go/No-Go)

P0 (must before production)

- CloudFront + WAF in front of web + API
- App-level rate limiting (IP + user + API key)
- Provider stoplist + circuit breaker enforced by scheduler
- Gold Publisher gates enforced (N≥3, dominance, rounding/suppression)
- Aurora backups + restore drill completed at least once
- SES bounce/complaint suppression integrated
- Stripe webhook idempotency + reconciliation path
- Admin audit logging for all high-risk ops actions

P1 (strongly recommended)

- Canary deploys + rollback automation
- Data quality checks (anomaly guardrails) integrated into pipeline
- Cost guardrails (concurrency caps, budgets, alarms)
- “Derived boundary tests” in CI (unit + integration)

💸 Advertisements & Affiliate Links

Goal: Maintain a clean, trustworthy user experience while supporting the business.

What we do not do

- No pop-ups or intrusive ad placements.
- No third-party ad networks or ad scripts running on our site.

Sponsored Listings (Admin-Curated)

- Remit-Scout supports **first-party sponsored placements** curated by authorized admins.
- **Clear labeling:** every sponsored placement must be labeled “Sponsored”.
- **No ranking impact:** sponsorship **never** affects organic ordering or scoring.
- **No third-party ad networks:** we do not run generic ad scripts/banners/popups; sponsorship is platform-native and limited.

Rights & Eligibility

Sponsored providers must still satisfy **Data Rights Matrix** constraints:

- **Allowed_B2C must be true**
- Sponsorship does not override any provider display constraints

Affiliate programs

We partner with some service providers through affiliate programs. This means certain outbound links to provider websites (e.g., money transfer services we compare) may include a referral parameter/code.

If a user clicks an affiliate link and later signs up or transacts, we may receive a small commission.

Our commitments

1) Clear disclosure

- We label or otherwise indicate affiliate links where required.
- We aim to be transparent so users understand there may be a commission.

See: Affiliate Disclosure Guidelines

2) No bias in rankings

- Affiliate relationships **do not** influence comparison results or rankings.
- Providers are listed based on product criteria (rates, fees, etc.), not affiliate status.

See: Ranking & Neutrality Policy

3) User privacy

- Clicking an affiliate link navigates the user to the provider's site.
- Referral tracking is typically handled on the provider's side (URL parameters and/or provider-domain cookies).
- We do **not** share user PII with affiliate partners during a click.
- We may receive **aggregate** conversion stats (e.g., totals), not individual identities.

Why we restrict advertising to affiliate links only

By avoiding third-party ad networks:

- **Faster load times** (no heavy ad frameworks)
- **Less clutter**
- **Greater privacy** (no widespread profiling via ad scripts)

Implementation notes

- Ensure outbound affiliate links are handled via a consistent mechanism:
  - e.g., `outbound_redirect` service or tracked redirect endpoint
- Ensure disclosure requirements are met in UI and in Privacy Policy.
- Ensure internal telemetry does not capture PII from outbound URLs (sanitize).
- Sponsored placement is a **separate display block** (or clearly separated in list)
- Organic list remains pure (sorted by product criteria only)
- Sponsored config stored in a controlled table; editable only via admin dashboard; all changes logged

Checklist for changes

If we add a new affiliate partner or change link handling:

- Add/update partner in Affiliate Partner Inventory
- Confirm disclosure placement in UI + policy text
- Confirm ranking logic remains independent
- Confirm telemetry sanitization is still correct
- Update this page + Cookies & Privacy if cookie behavior changes

📊 Analytics & Tracking Tools

Purpose: Use analytics to improve the product and measure marketing effectiveness while respecting privacy and user choice.

Principles

- **Privacy-conscious configuration** by default
- **No PII sent** to analytics/ads tools
- **Opt-out supported** where required (cookie banner/settings)
- **Aggregate-first** reporting (avoid user-level analysis unless needed for troubleshooting)
- **No selling or trading** analytics data

Tools we use (and why)

1) Google Analytics

Use: Aggregate usage insights (page views, time-on-page, journeys).
Privacy configuration:

- IP anonymization where possible
- No PII sent (no names/emails/etc.)

Outcome: Identify popular pages and drop-off points to improve UX.

2) Meta Pixel (Facebook Pixel)

Use: Ad conversion measurement for Facebook/Instagram campaigns.
Notes:

- Tracks actions after an ad click (page views, key button clicks)
- Uses pseudonymous browser identifiers
- We do not intentionally send sensitive personal data into the pixel
- Users can opt out via platform settings and/or our cookie preferences (jurisdiction dependent)

3) Other tracking pixels & tags (as needed)

Examples (only when we run campaigns):

- Google Ads conversion tag
- LinkedIn Insight Tag

Optional UX tools (only if enabled and approved):

- Heatmaps (e.g., Hotjar, CrazyEgg)
- Session replay tools (if used, must be reviewed carefully)

4) Internal telemetry (product analytics)

We also maintain a privacy-safe internal telemetry system for product events such as:

- Search queries
- Outbound link clicks
- In-app actions

Data minimization:

- Non-identifying IDs only (avoid direct identifiers)
- Strict gating/sanitization to reduce risk of sensitive data capture
- Stored internally (e.g., Silver layer) and not shared externally
- Access restricted to the team

Tracking inventory (recommended table)

(Keep this table up to date.)

Tool	Category	Purpose	Typical events	PII allowed?	Consent required?	Owner
Google Analytics	Analytics	Site usage	Page views, engagement	No	Yes (where required)	Growth
Meta Pixel	Advertising	Conversion attribution	Page view, key actions	No	Yes (where required)	Growth
Internal Telemetry	Product analytics	Feature usage	Searches, outbound clicks	No	Depends on jurisdiction/ policy	Eng

Link: Tracking & Tag Inventory

User controls & compliance

- Cookie banner shown where legally required (e.g., EU/UK).

- Users can change consent later via Cookie Preferences.
- We aim to respect “Do Not Track” signals where feasible.
- If a tool could associate behavior with an ad profile (Meta/Ads tags), we disclose and support opt-out.

Checklist for adding or changing a tracking tool

- Add the tool to Tracking & Tag Inventory
- Confirm it is configured with privacy best practices (no PII)
- Confirm consent gating (cookie banner + preferences)
- Update Cookies & Privacy and Privacy Policy
- Update CSP/security review if adding new scripts
- Validate telemetry/events are sanitized

🍪 Cookies & Privacy

Purpose: Explain what cookies we use, why we use them, and how users control them.

Types of cookies

Essential cookies

Required for core functionality (e.g., login sessions).

- Typically first-party
- Should contain no sensitive PII (randomized tokens)
- Usually exempt from opt-in requirements (jurisdiction dependent)

Preference cookies

Remember user preferences (e.g., language, home currency) if applicable.

Analytics cookies

Used by analytics tools (e.g., Google Analytics) for aggregate measurement.

- Should not store direct personal details
- Use reasonable expiration settings

Advertising / tracking cookies

Used by ad conversion tools (e.g., Meta Pixel) if enabled.

- Only deployed with consent where required
- Users can manage via cookie preferences and platform settings

---

Cookie consent & control

- When required, we show a cookie consent banner on first visit.
- Users can accept/decline non-essential cookies.
- Users can adjust preferences later via Cookie Preferences.
- Site should still function even if optional cookies are declined (with reduced optional functionality).
- We strive to respect “Do Not Track” signals where feasible.

Data privacy practices

Minimal data collection

We collect/store personal information only as necessary to provide the service.

No selling of personal data

We do not sell or rent user personal data to third parties.

Disclosure & consent

We disclose what we collect and why in Privacy Policy.
Where consent is needed (marketing emails, optional cookies), we request it explicitly.

Data security

- Encryption in transit (HTTPS/TLS)
- Encryption at rest where appropriate
- Strong access controls
- Passwords never stored in plaintext (handled by auth provider)

Retention & deletion

- We retain personal data only as long as necessary.
- Account data retained while account is active.
- On deletion request, we erase or anonymize PII where appropriate, except where legally required to retain records.

Link: Data Retention Policy and Account Deletion Process

Third-party privacy

If users click affiliate links or interact with third-party services, those third parties govern their own data handling under their privacy policies. We aim to choose reputable partners and minimize shared data.

Data subject rights (by jurisdiction)

Users may have rights such as:

- Access & portability
- Rectification
- Deletion
- Opt-out of marketing
- Cookie consent controls

- **Consent model**: necessary vs analytics vs marketing cookies
- **Default:** only necessary cookies until consent
- **Do Not Track / opt-out** handling (if you support it)
- **Data minimization:** avoid IP storage where possible; anonymize session IDs
- **Affiliate disclosure + sponsored disclosure** tie-in

🔑 Supabase Configuration Considerations

Purpose: Document how we use Supabase for authentication and how we keep it secure.

1) Role in our architecture

Supabase functions as our **Identity/Auth store** (separate from our main application DB). We keep:

- Auth credentials and minimal user identity data in Supabase Postgres
- Core product and operational data in our main data plane (e.g., Aurora “Silver”)

Our backend services verify Supabase JWTs to authenticate requests. To reduce latency and dependency on frequent external calls, we cache Supabase JWKS (public signing keys) server-side.

2) Secure configuration & keys

We treat Supabase secrets with appropriate sensitivity:

Key / Setting	Where used	Sensitivity	Storage approach	Notes
Supabase URL	Frontend/backend config	Low	Config/env	Not a secret
Supabase Anon Key	Frontend	Low (public-by-design)	Config/env	Should not grant admin access
Supabase Service Role Key	Backend only	**High**	Secrets manager / secure env	Never exposed client-side
JWKS (public keys)	Backend verification	Public	Cached server-side	Refresh periodically

Environment separation

- Separate Supabase projects for dev/staging/prod
- Prod keys only in deployment environment (not on dev machines)

Rotation

- Rotate sensitive keys (especially service role key) on schedule or on suspicion of compromise

3) Access controls and policies

- Use Row-Level Security (RLS) to ensure users can only access their own data.
- Default-deny posture: tables are not readable/writable without explicit policy.
- Admin access restricted (MFA, limited team membership).

4) Data handling in Supabase

- Store only what's required for authentication (minimal PII).
- Avoid storing unnecessary PII in Supabase; keep application-specific profile data in the main app DB if needed (with appropriate safeguards).

5) Integration guardrails

- Backend communicates with Supabase over HTTPS only.
- Frontend does not "directly query everything"; most operations flow through our backend APIs for validation and control.

6) Monitoring and logging

- Monitor authentication events and suspicious patterns (e.g., repeated failed logins).
- Ensure rate limiting is configured appropriately.
- Document incident response steps: Auth Incident Runbook

Checklist for changes

- Confirm key exposure rules (service role key never client-side)
- Confirm RLS policies updated and tested
- Confirm JWKS caching/refresh behavior still correct
- Update this page + Secrets Management

🔐 Security Architecture Overview

Purpose: Summarize our security posture and the architectural guardrails that keep data compartmentalized and systems resilient.

Core Security Principles

- **Separation of Concerns (Layered Planes):** We divide responsibilities across three planes so that raw internal data is isolated from user-facing surfaces.
  - **Plane A (Product Delivery):** User-facing APIs and website (the app delivery layer).
  - **Plane B (Ingestion Engine):** Data collectors and processors that gather raw provider data and normalize it.
  - **Plane C (Analytics & Publishing):** Aggregators that derive insights and publish **only** processed outputs.
    This separation ensures that raw provider data (Bronze) remains inaccessible from user-facing services, even in the event of bugs or incidents.
- **Least Privilege:** Every service and role is granted the minimum access required to perform its function. This is enforced via strict IAM policies, scoped credentials, and tightly-configured security groups.
- **Network Segmentation:** Sensitive resources reside in private subnets with no direct internet ingress. Public access is limited to essential entry points (e.g. API Gateway, CloudFront) and protected by WAF and other controls.
- **Strong IAM Boundaries:** IAM acts as the central gatekeeper between planes. For example, by policy **Plane A cannot access Bronze** (raw storage). Cross-plane access is explicitly blocked unless permitted by design.
- **Auditability:** All data movement and sensitive actions are logged and traceable. We leverage CloudTrail and other logging to ensure every important operation (especially data access or privilege changes) can be audited end-to-end.
- **Fail-Safe Defaults:** If a security check fails or the system is unsure, the default behavior is to **deny access, stop the action, or withhold data**. We favor safety over availability in ambiguous situations (e.g. if a data classification is uncertain, we don't publish it).
- **Defense in Depth:** Multiple layers of controls overlap to protect the system even if one layer falters. For example, IAM restrictions, network ACLs, and application-layer validations all work

together. Additionally, all data is encrypted in transit and at rest, dependencies are regularly patched, and infrastructure is configured following secure best practices.

Security Deep-Dive Pages

The following pages provide a deeper technical look at our security architecture and operations:

- ☁️ **AWS Account & Network Topology** – How our AWS account structure and VPC networking enforce isolation and reduce blast radius.
- 🪪 **IAM Roles Catalog** – A catalog of IAM roles, detailing which plane each operates in, what each role is allowed to do, and what it is forbidden from doing.
- 🔁 **Access Matrix** – A matrix of data stores vs. planes, clarifying which planes/services can read or write to each storage layer (Bronze, Silver, Gold, etc.), and highlighting the strict data access boundaries.
- 🤫 **Secrets Management** – Our approach to storing, accessing, rotating, and auditing secrets (API keys, credentials), including where secrets live and how we protect them.
- 🧪 **Boundary Tests** – The tests and checks we continuously run to verify that security boundaries hold (e.g. raw data never leaks to Plane A, ingestion stops on block signals, IAM boundaries are in place), as well as other security validation and monitoring practices.

☁️ AWS Account & Network Topology

Purpose: Document how our AWS account structure and networking are configured to enforce isolation and reduce blast radius.

AWS Accounts Structure

We use multiple AWS accounts to separate environments and limit the impact of any single account compromise:

- **Production Account:** Hosts the live, customer-facing environment. It has the most restrictive access controls and closely monitored actions.
- **Staging/Testing Account:** A safe proving ground for new changes and integration testing. It is isolated from production resources to prevent any spillover.
- **Development Account (or Dev Resources):** Used for day-to-day development and experimentation. This isolation ensures that testing or prototype work cannot affect production.
- *All accounts are organized under AWS Organizations with guardrails in place (for example, mandatory CloudTrail logging on all accounts for auditing).*

VPC Topology (Production)

Within the production account's VPC, network segmentation is used to protect sensitive components:

- **Private Subnets:**
  These subnets host sensitive resources such as databases (Aurora), internal compute (Lambdas/ECS tasks for Plane B and C), and analytics services. They **do not allow any direct inbound internet traffic**. Outbound internet access from these subnets is only possible via a NAT gateway as needed for updates or external calls. By keeping these resources in private subnets, we ensure they are not directly reachable from the public internet.
- Plane B collectors may route outbound HTTP(S) through an approved **residential proxy network** to achieve localization and improve reliability.
- Proxies are treated as external endpoints reachable via NAT; **no inbound access is introduced**.
- Security groups allow egress only to:

    - proxy vendor endpoints (control plane / gateways)
    - destination provider domains (if direct allowed for some sources)
- Logging: per-run proxy metadata captured (provider, region, exit country).
- Using proxies does not change plane isolation. Plane B still cannot reach Plane A private services, and Plane A still cannot read Bronze.
- **Public (Edge) Subnets:**
  These subnets are used for essential outward-facing services like API Gateway endpoints or a load balancer, as well as CDN ingress/egress (CloudFront with WAF). We minimize the resources placed here—ideally only stateless entry point services. Most application compute remains in private subnets, even if invoked via API Gateway. This design limits the exposure of compute resources and funnels public traffic through controlled chokepoints (like WAF and API Gateway).

Admin Access Approach

For administrative access to servers or internal infrastructure, we favor secure, no-exposed-port methods:

- **Preferred Method – AWS SSM Session Manager:** We use AWS Systems Manager Session Manager to gain shell access to EC2 instances or other resources in private subnets. This allows admin access **without opening any inbound SSH ports** or running bastion hosts. Session Manager connections are logged and controlled via IAM, providing both security and auditability.
- **Optional Bastion (Discouraged):** In rare cases where a bastion host is necessary, we use a **hardened bastion** instance. This host is strictly locked down to a specific allowlist of IPs, used only when absolutely needed, and shut down or disabled when not in use. All access via the bastion is monitored. (Our goal is to avoid bastions if possible by using Session Manager and other tools.)

Routing and Security Groups

We design network routing and security group rules to enforce least privilege at the network level:

- **Database Access Controls:** For example, the database security group for Aurora (Silver) allows inbound connections **only** from approved application instances or Lambda security groups in the same VPC. No public ingress is allowed, and other services cannot reach the database unless explicitly required. This ensures that even within our AWS environment, only the application components that should talk to the database can do so.

- **Outbound Restrictions for Ingestion:** Plane B ingestion services (collectors) often need outbound internet access to gather data from external providers. We permit those outbound calls, but we tightly restrict any **east-west (internal lateral)** access they have. Collectors do not have broad access to internal services beyond what's needed (they typically just write to the Bronze storage). This minimizes the impact if a collector service were compromised or misbehaving.

Data Layer Networking

Each data storage layer is isolated and accessible only to the appropriate planes and services:

- **Bronze S3 Bucket (Raw Data Lake):** This bucket is completely private—no public access, no web hosting. We typically configure a VPC Endpoint for S3, so when internal services (collectors in Plane B) write to or read from Bronze, that traffic stays within AWS's private network and doesn't traverse the public internet. Plane A services have no credentials or network path to this bucket by design.
- **Silver (Aurora Database):** The Aurora database instance for Silver has **no public endpoint**. Only the application and ingestion services within our VPC (Plane A and B roles that need it) can connect, and those are controlled via security groups and IAM auth. Silver contains operational data derived from Bronze, and is treated as sensitive (not directly exposed to end-users without going through Plane A APIs).
- **Gold (Analytics Store – e.g. ClickHouse):** The Gold data store is also hosted in private subnets with restricted access. Only Plane C processes and specific analytic queries (via our APIs) can reach it. It's not exposed publicly. When we generate export files or serve analytical queries from Gold, it's done through controlled services that apply business rules (such as data aggregation thresholds).
- **Supabase (Auth DB):** Our Supabase Postgres (for authentication data) is a managed service but similarly we treat it as an internal component. Frontend and backend communicate with it through secure channels. We ensure any direct connections to it come from approved origins or via our backend, not from arbitrary public sources.
- **Redis Cache:** If we use a Redis cluster for caching/rate-limiting in Plane A, it is placed in a private subnet. Only the application Lambdas or services that need the cache can connect. It's not publicly accessible and often configured with TLS and auth tokens for an extra layer of protection.

Account-Level Security Services

At the account level, we enable several AWS security services to monitor and guard the environment:

- **CloudTrail:** Enabled organization-wide to log **all API calls and console actions** in each account. These audit logs are stored securely, providing an audit trail for any changes or access to resources. CloudTrail is fundamental for forensic analysis in case of an incident.
- **GuardDuty:** Enabled on all accounts to provide intelligent threat detection (e.g. alert on anomalous API calls, suspected instance compromises, crypto mining, etc.). GuardDuty findings are integrated into our alerting/SIEM so that security issues are quickly noticed and addressed.
- **AWS Config Rules:** We implement AWS Config with rules to detect misconfigurations or drifts from our security baseline. For example, rules ensure that no S3 buckets are publicly readable, that IAM keys are rotated, and that security groups don't have overly broad access. If a rule is violated (e.g. someone accidentally opens an S3 bucket), it triggers an alert (and in some cases auto-remediation) to maintain our secure posture.
- **Automated Alerts on High-Risk Changes:** Changes to critical settings (like security group modifications, network ACL changes, IAM policy changes) generate notifications. This way, if an administrator or pipeline modifies something that could weaken security, the team is immediately aware and can review if it was intended and safe.

🪪 IAM Roles Catalog

Purpose: Maintain a living catalog of IAM roles in our system, including which plane each role operates in, what it is allowed to do, and what it is explicitly forbidden from doing. This ensures clarity on access control and helps with reviewing permissions over time.

We keep this catalog aligned with our Infrastructure-as-Code definitions (CDK/Terraform), and each role entry links to its actual IAM policy source for reference. This catalog and the linked policies should be reviewed regularly to uphold least privilege.

Below is the list of key IAM roles and their access scopes:

Role	Plane	Used By	Allowed Actions (High-Level)	Explicitly Not Allowed
API Lambda Role	A	API Handlers (Plane A)	Read from Silver (operational DB); read from Gold export store; enqueue export jobs to queue; send notifications (e.g., via SES/SNS)	**No** access to Bronze S3 (raw data); Cannot assume any higher-privilege roles.
Web Frontend Deploy Role	A	CI/CD Pipeline (Deploy)	Upload static assets to S3; Invalidate CloudFront cache (for frontend)	**No** access to application data stores (no DB or S3 data access).

Collector Role	B	Data Collector Lambdas/Tasks (Plane B)	Write to Bronze (raw data S3); Write normalized data to intermediate storage as designed (e.g., insert into Silver via ingest process) 1. `secretsmanager:GetSecretValue` for **only** the proxy secret ARN 2. outbound egress to proxy vendor endpoints (documented in network rules)	**No** access to Gold or export data; **No** access to any Plane A resources or services. 1. Any Gold read permissions 2. Any Plane A internal access 3. Any “admin override” permissions
Normalizer/ Processor Role	B	Data normalization jobs (Plane B)	Read from Bronze (as needed for processing);	**No** Plane A access; **No** access to Gold tier.

			Write cleaned data to Silver (Aurora)	
Aggregator Role	C	Batch aggregation jobs (Plane C)	Read from Silver (to aggregate data); Write to Gold internal store; After gating checks, write to Gold export (derived outputs)	**No** access to Bronze; **No** unrestricted external egress (only whitelisted export endpoints).
Export Worker Role	C	Export generation service (Plane C)	Read from Gold export store; Write generated export files to S3; Generate signed URLs for download	**No** direct reads from Silver (operates only on already-derived Gold data by policy); **No** access to Bronze.
Admin / Break-glass Role	N/A	Privileged humans (on-call admins)	Broad maintenance capabilities for emergency use (database access,	Not used by any application runtime service (human use only); Intended for

			infrastructure changes) – usage is heavily logged/monitored	emergencies **only**.
Read-only Audit Role	N/A	Auditors / automation tools	Read access to configuration and security posture (e.g., view IAM policies, Config, CloudTrail logs)	**No** ability to modify resources or data; cannot write or change settings.

*Each role's full policy document is stored in our repository (see the **IAM Policy source** in code) and changes to any role's permissions require review (see the **IAM Review Checklist** for the process to request or approve permission changes). Keeping this catalog up-to-date is part of our security review process.*

🔁 Access Matrix

Purpose: Make our data access boundaries explicit by showing which planes/services can access which data stores. This matrix reinforces the rule that raw data remains confined to ingestion layers and only processed data is available to the user-facing plane.

The table below summarizes access by data store and plane:

Data Store	Plane A (Product)	Plane B (Ingestion)	Plane C (Publishing)
Bronze (Raw data S3)	**None.** Plane A has no access to raw data.	**Write** (collectors write raw data); **Read (limited internal)** for processing within B as needed.	**None.** Plane C does not touch raw data.
Silver (Operational DB - Aurora)	**Read/Write (Scoped).** Plane A services can read and write current operational data (through APIs) with fine-grained scope (e.g., only their data or permitted tables).	**Write (Ingestion).** Plane B normalization processes write cleaned data into Silver.	**Read (Aggregation).** Plane C analytics jobs read Silver as input for deriving Gold data.
Gold (Analytics Store - e.g. ClickHouse)	**Read (Exports Only).** Plane A can only read from Gold via controlled export endpoints (only aggregated, derived data that passed publishing rules).	**None.** Plane B does not interact with Gold.	**Read/Write (Internal).** Plane C writes to Gold internal tables and reads them to produce exports; Gold exports are the only outputs exposed outside.
Supabase Auth DB (User auth data)	**Read/Write via Auth APIs.** Plane A (our API/website) interacts	**None.** Ingestion has no need for auth DB.	**None.** Publishing plane has no need for auth DB.

	with Supabase for authentication and user data through secure APIs.		
Redis Cache (ephemeral cache)	**Read/Write.** Plane A uses cache for rate limiting, sessions, etc., as needed.	*Usually none.* (Plane B might use its own caches internally if ever needed, but generally not shared with A.)	**None.** Plane C doesn't use the web cache.
Export Bucket/Files (Derived data exports)	**Read (Signed URLs).** Plane A (and ultimately end-users) can read export files via pre-signed URLs generated by Plane C.	**None.** Ingestion doesn't access exports.	**Write (Export Worker).** Plane C's export workers write finalized export files to this bucket (and generate read URLs).

Invariants / Rules:

- Plane A (user-facing) **must never access Bronze** raw storage directly. There are no credentials or network paths that would even allow it by design.
- Plane C (publishing) **only publishes derived data** that has passed all export safety checks. It never exposes low-level records that haven't been aggregated or vetted.
- Any exception to these access rules would require an explicit architecture decision (ADR) and a security review. To date, we maintain strict adherence to this matrix to uphold the "Golden Rule" that **raw data never leaves Plane B**.

(These boundaries are enforced via IAM policies, VPC configuration, and application logic. The Access Matrix should be updated if any new data stores are added or if roles change access patterns.)

🧪 Boundary Tests

Purpose: Describe the tests and checks we perform to continuously verify that our security and compliance boundaries are intact. These “boundary tests” ensure that data stays in the right places and that our guardrails work in practice, not just in theory. They range from automated tests in code to periodic exercises and monitoring alerts in production.

1. Golden Rule Enforcement Tests (Raw Data Never Leaks)

We regularly test and audit to ensure that **Plane A never receives raw data** from Plane B:

- **API Response Audits:** We inspect responses from user-facing APIs to confirm that they only contain data from Silver or Gold (processed data). For example, if an API is supposed to return current exchange rates, we verify those rates came from the Silver DB or a Gold aggregate, and not directly from raw Bronze records.
- **Export Data Validation:** We test the data files generated for export (by Plane C) to ensure they comply with publishing rules. This includes verifying that aggregation thresholds are met (e.g., no dataset has too few data points that might reveal individual raw values) and that any required suppression (like removing provider names or PII) is in place.
- **Access Rejection Tests:** We deliberately attempt operations that should be disallowed (such as a Plane A function trying to read from the Bronze S3 bucket or a direct query for raw data through an API) in a controlled test environment. These attempts should consistently fail with access denied errors, proving that IAM and application logic are effectively enforcing the data firewall.

2. Stop-on-Block & Compliance Tests (Ingestion Resilience)

Our ingestion engine (Plane B) is designed to **fail safely** if it encounters roadblocks or compliance issues. We simulate these scenarios to verify the correct behavior:

- **Blocked Source Simulation:** We test what happens when a data source starts returning HTTP 403 Forbidden or presents a CAPTCHA/human verification. In our integration testing, we simulate such responses and ensure the collector immediately **stops** trying to fetch data from that source. The expected behavior is to cease collection and escalate an alert for human review, rather than attempting to circumvent the block.
- **Stoplist and Circuit Breaker Tests:** We maintain “stoplists” (providers or data we must not collect) and circuit breaker limits (caps on how much or how often we collect). Automated

tests feed dummy entries or threshold breaches to the ingestion process to confirm that it **halts or skips** those entries. For example, if a provider is marked as disallowed, any job for it should be skipped entirely. If a certain corridor's collection frequency exceeds its budget, the system's circuit breaker should pause further collection.

- **Rights Matrix Alignment:** We cross-check ingestion against our **Compliance & Rights Matrix** (a separate document outlining what we are allowed to collect from each provider). Tests ensure no collector runs for a provider or data type that has not been approved. If a developer accidentally introduces a new provider integration without proper approval, our review process and tests catch it before it goes live.
- **Simulate** CAPTCHA/403/429/anti-bot page while proxy is enabled
- Assert:
  - collector stops immediately
  - circuit opens
  - stoplist moves to Paused (or triggers escalation workflow)
  - does **not** cycle IPs to continue
- **Assert collectors only use approved proxy endpoints (reject unapproved proxy host config)**

3. Penetration Testing & Threat Simulations

Periodically, we undertake exercises to simulate real-world attacks and ensure our defenses hold up:

- **External Pen Tests:** We engage independent security professionals to perform penetration testing on our platform. They probe for vulnerabilities in our APIs, web application, and cloud infrastructure. Findings from these tests are used to harden our systems. We document and fix any weaknesses discovered, and update our security practices accordingly.
- **Internal Threat Simulations:** We conduct drills such as simulating a compromised credential or an inside actor. For example, we might simulate what would happen if an AWS access key leaked. These tests confirm that our monitoring (GuardDuty, CloudTrail alerts) would detect unusual behavior (like data exfiltration or privilege escalation) and that our incident response playbooks are effective.
- **AWS Config & GuardDuty Continuous Assurance:** While not a one-time test, our use of AWS Config rules and GuardDuty is essentially an ongoing "test" of our environment's security. Any time someone tries to, say, open an S3 bucket to the public or deploy an instance in a public subnet against policy, AWS Config flags it. We treat these alerts as test failures that need immediate remediation, ensuring our baseline stays intact daily.

4. Automated Security Unit/Integration Tests

Security is also part of our software testing pipeline:

- **Authorization Unit Tests:** For each API endpoint and service, we have tests to ensure that role-based access control is correctly enforced. For example, a regular user should not be able to access admin-only endpoints or another user's data. We write tests for various roles (user, admin, support) to verify that permissions are correctly implemented in code.
- **Row-Level Security (RLS) Tests:** Given that we use Supabase for auth and possibly row-level security for multi-tenant separation, we include tests that ensure one user cannot read another's records. If RLS policies are in place on certain tables, we add integration tests that attempt cross-tenant data access and assert it fails.
- **Infrastructure-as-Code Policy Tests:** Our infrastructure code (IaC) is tested to ensure security configurations. We use tools or custom scripts to assert, for instance, that a Plane A role has no permission on a Bronze bucket. If someone updates a Terraform file that inadvertently grants broader access, a unit test should catch that before it's applied. We also test that security group rules remain restricted (no 0.0.0.0/0 on sensitive ports, etc.).

5. Deployment Security Checklist (Human Gate)

Beyond automated tests, any deployment or architectural change that could affect security must go through a manual checklist review:

When preparing to ship a change, we ask:

- **Which Planes/Services Are Affected?** Identify if the change introduces a new service or alters an existing one in Plane A, B, or C. We then verify the relevant security assumptions for that plane. For example, if it's a new Plane A feature, ensure it doesn't need any forbidden data access.
- **IAM Permissions Review:** Check that any new or modified IAM roles/policies do not broaden access in unintended ways. We compare the new IAM policy against our least-privilege guidelines. If a Lambda function gets additional permissions, are they absolutely required? This often involves code review of IaC diffs and sometimes running IAM policy analysis tools.
- **Network Exposure Review:** Confirm that no new network paths are introduced that violate our segmentation. For instance, if a new service needs internet access, we ensure it's placed correctly (likely in a private subnet with NAT, unless absolutely requiring public exposure). If a new port is opened on a security group, we double-check the necessity and scope.
- **Monitoring/Alerting:** Ensure that if the change involves new infrastructure or data flows, our logging and monitoring cover them. For example, if we add a new S3 bucket, is CloudTrail

logging access to it? Is it included in Config rules (no public access)? Do we need to create a new alarm for high error rates or unusual activity for that component?

- **Documentation Update:** We verify that this security documentation (Access Matrix, IAM Catalog, etc.) is updated if needed. For example, adding a new IAM role means adding it to the IAM Roles Catalog. Changing a data flow might require an update to the Access Matrix or an ADR note if it's an exception to an existing rule.

Each item on this checklist must be acknowledged by the engineer and reviewed by a peer or security champion before deployment proceeds. This human-in-the-loop process acts as a final gate to catch anything automation might miss.

6. Continuous Monitoring as Testing

Finally, we treat our production monitoring and alerting as a form of continuous security testing:

- **Alert on Access Denied Patterns:** We have alerts set up for repeated AWS `AccessDenied` errors in logs. If a service or user is seeing a lot of denied requests, it could indicate either a misconfiguration (less likely, since that would be caught in testing) or a probing/directory traversal attempt by a malicious actor. For instance, if someone tries to call a Plane B-only API from Plane A, it should be denied and trigger an alert.
- **GuardDuty Findings and Triage:** AWS GuardDuty runs continuously and flags potential issues (port scans, anomalous logins, etc.). We treat each GuardDuty finding as if it were a failed test: our incident response runbook is triggered, and we investigate and resolve the issue. This ensures that if any unexpected activity occurs, it's dealt with promptly and the root cause (if a control failed) is identified and fixed.
- **CloudTrail and Audit Logs:** We regularly examine CloudTrail logs for any irregular access patterns. For example, if an IAM role that normally isn't used out of hours suddenly is active at 3 AM, we investigate. We also use automated tooling to scan for changes like "who last changed this security group" or "which users have recently escalated privileges". This ongoing audit process catches lapses that might have slipped by initial reviews.

All these tests and monitoring activities form a feedback loop. When something is caught (be it in a simulated test or real monitoring), we not only fix the immediate issue but update our processes or controls as needed. This could mean adding a new unit test, adjusting an IAM policy, improving an alert threshold, or providing additional team training. Our goal is to continually **verify and strengthen** the security boundaries that protect our system.

🤫 Secrets Management

Purpose: Define how we store and handle sensitive secrets (credentials, API keys, tokens), including our practices for secure storage, access control, rotation, and audit. Effective secrets management prevents leaks and limits exposure in case of credential compromise.

Where We Store Secrets

- **AWS Secrets Manager:** This is our primary store for sensitive secrets such as database credentials, API keys for third-party services, and any high-impact secret. Secrets Manager provides encryption at rest (via KMS), controlled IAM access, and automated rotation capabilities for supported secret types. We make use of these features where possible (e.g., auto-rotating RDS credentials).
- **AWS SSM Parameter Store (SecureString):** We use Parameter Store for certain configuration values and less critical secrets where appropriate. SecureStrings are encrypted with KMS as well. This is often used for parameters that our applications need (like a specific API endpoint or low-sensitivity tokens) that still should not be in plaintext, but which benefit from simpler retrieval via the EC2/Lambda environment.

No secrets are stored in code repositories or in plain text on disk. All application secrets reside in one of the above secure stores, never hard-coded in source code or configuration files that could be exposed.

Key Rules and Practices

- **No Hard-Coded Secrets:** Developers must never commit secrets or passwords to the codebase. Code reviews include checks for this, and automated scanners run on our repos to catch any accidental commits of secrets.
- **No Logging of Sensitive Data:** Application code is structured to avoid printing secrets or personal data to logs. This prevents accidental exposure via log files or monitoring systems. For instance, when catching an error connecting to the database, the connection string (with password) is never logged.
- **Strict Access Control:** Access to retrieve secrets from Secrets Manager or Parameter Store is tightly controlled by IAM. Each service or Lambda function only has IAM permissions for the specific secrets it needs. For example, the Plane A API Lambdas can only retrieve the

database credential for Silver and perhaps a limited set of API keys needed for their features—nothing else. This principle limits blast radius if any single service is compromised.

- **Audit and Monitoring:** Every access to a secret is logged by AWS (CloudTrail events for Secrets Manager and Parameter Store access). We monitor these logs and have alerts for unusual access patterns (e.g., a secret being read by an identity that typically doesn't access it, or secrets being accessed at odd hours in bulk). This helps detect potential leaks or misuse.

Secret Rotation Policy

Regular rotation of secrets mitigates the risk of long-lived credentials. Our rotation practices include:

- **Database Credentials:** Whenever possible, we use AWS's IAM authentication for databases (like Aurora) to avoid static passwords. If static credentials are used, they are rotated on a schedule (e.g., every 90 days) or immediately if any suspicion arises. We automate rotation via Secrets Manager for RDS credentials where supported, which updates the credential and notifies our applications (or auto-reloads).
- **Supabase Service Role Key:** The service role key for Supabase (which grants elevated access to the auth DB) is treated as highly sensitive. We rotate this key on a regular schedule (and immediately if we suspect any issue). Rotation involves generating a new key in Supabase and updating it in Secrets Manager, with a coordinated deployment so that applications use the new key.
- **Third-Party API Keys:** API keys for providers (e.g., Stripe, data providers, etc.) are rotated per the provider's recommendations or sooner if needed. If a provider offers multiple keys or roll-over capabilities, we take advantage of that to ensure we can change keys without downtime. We also try to scope third-party keys to only necessary permissions on the provider side (for example, using separate keys for different services to avoid over-permission).
  - **Residential Proxy Service Key** is stored in **AWS Secrets Manager**.
  - Accessed by **Plane B collectors only** (collector role).
  - Rotated per vendor policy or immediately on suspicion.
  - Retrieval is audited via CloudTrail; alert on abnormal access volume.
- **Key Rotation Procedures:** We maintain runbooks for how to rotate each type of secret. These runbooks detail the steps and any potential impact (for instance, how to update an API key and test that everything still works). Automated alerts remind us of upcoming rotation dates for keys that have a known lifespan.

Encryption and Handling

- **Encryption in Transit:** Any time a secret is transmitted (from Secrets Manager to an application, or from our service to a third-party), it is done over TLS-secured channels. This prevents eavesdropping on credentials in flight.
- **Encryption at Rest:** Secrets Manager and Parameter Store both rely on AWS KMS to encrypt data at rest. Additionally, any secrets cached or stored temporarily by our applications (e.g., in environment variables or memory) reside on encrypted disks by virtue of running on AWS services (such as encrypted EBS volumes for EC2 or the ephemeral storage for Lambda).
- **Limited Lifetime in Memory:** Applications retrieve secrets at startup or on-demand and avoid persistent storage. For example, a Lambda function might fetch a database password from Secrets Manager when it initializes, use it for the duration of the run, and then it's gone when the function context is destroyed. We avoid writing secrets to local files. If an application needs to cache a secret (to avoid repeated Secrets Manager calls), it's kept in memory only, not written to disk.
- **Secure Secret Updates:** When updating or deploying infrastructure, we use AWS CDK/Terraform scripts that reference secrets by name (never in plaintext). Updates to secrets themselves (the values) are done through the AWS console or CLI interacting with Secrets Manager, not through code commits.

Example: Secret Inventory

We maintain a confidential inventory of all our secrets with metadata about each one. An illustrative excerpt of such an inventory might look like:

Secret	Stored In	Accessed By	Rotation Schedule	Notes
Aurora DB Credentials	AWS Secrets Manager	API Lambdas (Plane A), Ingestion jobs (Plane B) as needed	Every 90 days (automated via AWS rotation)	Master credentials are rarely used; app uses IAM auth where possible.
Stripe API Secret Key	AWS Secrets Manager	Billing service (Plane A)	Rotated if compromised or annually	Not exposed on frontend; limited to

			(whichever is sooner)	billing operations.
Supabase Service Role Key	AWS Secrets Manager (backup in SSM)	Auth service backend (Plane A)	Every 60-90 days	High sensitivity; grants user management rights in Supabase.
Provider API Key X	AWS Secrets Manager	Specific Collector Lambda (Plane B)	Rotate per provider policy (e.g., annually) or on suspicion	Key is scoped to only necessary endpoints for that provider.
Residential Proxy API Key	Secrets Manager	Plane B Collector (Lambda/ECS)	Rotate every 60–90 days (or vendor)	Used for proxy gateway auth; never stored in code.

*(This table is maintained on an internal page. All team members must follow the **Secret Rotation Runbook** when changing any of these values. Every rotation is logged and, when possible, automated.)*